



A survey of anomaly detection in HPC systems using machine learning

Yiming Lu¹ · Wei Zhang¹ · Lihua Yang¹ · Yiqin Dai¹ · Huijun Wu¹ · Zhenwei Wu¹ · Hongyun Tian² · Yuqi Li¹ · Juan Chen¹ · Chubo Liu³ · Yong Dong¹

Received: 28 May 2025 / Accepted: 14 November 2025 / Published online: 16 January 2026
© The Author(s) 2025

Abstract

High-performance computing (HPC) systems must remain stable and reliable to consistently deliver robust computational power and ensure the proper execution of user jobs. Anomaly detection is a key means to ensure the stability and reliability of these systems. With the expansion of HPC systems and changes in their architecture, accurately identifying anomalies in dynamic environments has become increasingly challenging. Traditional detection methods rely on experience and rules, which could be inefficient and inaccurate. To address these issues, researchers have proposed machine learning-based methods to automatically process large amounts of complex data, improving the efficiency of anomaly identification and diagnosis. In this survey, we conduct a comprehensive and in-depth investigation of machine learning-based anomaly detection methods in HPC systems. Firstly, we summarize and introduce the background and challenges of anomaly detection in HPC systems. Secondly, we compare a series of machine learning-based anomaly detection works in detail and summarize their frameworks. We conclude their advantages and disadvantages and application scenarios. Finally, we discuss several promising development trends of machine learning-based HPC system anomaly detection.

Keywords High performance computing · Anomaly detection · Machine learning · Deep learning

1 Introduction

HPC systems are a type of computing architecture that achieves ultra-high performance by aggregating computing resources such as processors, storage, and networking. As societal development drives ever-increasing requirements for computational power, HPC systems are expanding in scale and architecture to meet the needs of modern scientific applications. The rapid advancement of HPC systems has

intensified concerns about system reliability. As faults and anomalies can lead to significant performance degradation or even system crashes, resulting in severe losses and compromised user experience. Moreover, as the scale of computing expands, the trade-off contradiction between system reliability and performance becomes a key bottleneck hindering the improvement of computing efficiency (Yang et al. 2012). However, relevant research shows that about 65% of abnormalities can be detected and avoided in advance by technical means to reduce the risks and losses brought by abnormalities (Oppenheimer et al. 2003). Meanwhile, compared to traditional fault tolerance and backup technologies in HPC systems, anomaly detection offers lower costs and greater flexibility. In summary, anomaly detection demonstrates more significant advantages in ensuring the long-term stability and reliability of HPC systems.

However, anomaly detection in HPC systems remains a challenging task. Firstly, the ever-increasing scale and component diversity of HPC systems have led to a significant increase in the frequency and diversity of anomalies. The Mean Time Between Failure (MTBF) of HPC systems has decreased from the order of hours to the order of minutes (Cappello 2009; Di et al. 2019). Secondly, HPC systems

✉ Yiming Lu
luyiming23@nudt.edu.cn

✉ Wei Zhang
zhangwei@nudt.edu.cn

✉ Yong Dong
yongdong@nudt.edu.cn

¹ College of Computer Science and Technology, National University of Defense Technology, Changsha, China

² Institute of Applied Physics and Computational Mathematics, Beijing, China

³ College of Computer Science and Electronic Engineering, Changsha, China

continuously generate vast amounts of high-dimensional, heterogeneous, and unlabeled data. Additionally, various types of noise within the system persistently interfere with anomaly detection results.

In recent years, with the improvement of system computational power and the rapid advancement of artificial intelligence, machine learning-based anomaly detection methods have been widely adopted across various domains in HPC systems, demonstrating superior detection performance and enhanced generalization capabilities. Against the backdrop of escalating failure costs (Lawrence and Simon 2023), machine learning-based anomaly detection methods have gradually evolved into a research focal point in the field of anomaly detection. The essence of machine learning lies in constructing mathematical models, optimizing parameters through training datasets, and ultimately enabling predictive capabilities for new samples. Compared to traditional experience and rule-based methods, machine learning-based anomaly detection methods can effectively integrate multi-source heterogeneous data and rapidly process high-dimensional data. Crucially, algorithms like deep neural networks autonomously extract nonlinear features and dynamically model spatiotemporal correlations in complex systems, significantly improving detection accuracy and inference capabilities. This empowers the models to better adapt to dynamic environments in HPC systems. Moreover, anomaly detection frameworks leveraging deep learning and natural language processing technologies substantially reduce learning curves for operators and minimize time spent on model deployment and fine-tuning. These attributes make machine learning-based anomaly detection methods particularly suited to real-world operational requirements in HPC system maintenance.

1.1 Contribution

This survey provides a comprehensive overview of machine learning-based anomaly detection methods on HPC systems. Firstly, we introduce the problem definition and background of anomaly detection in HPC systems. Secondly, we conduct an in-depth analysis of the challenges and the latest advancements in addressing these challenges. When discussing existing works, we focus on various machine learning methods along with their characteristics and applicable scenarios. Finally, we analyze several key development trends. Specifically, our contributions include the following:

- We conduct a comprehensive and extensive literature survey on the field of HPC system anomaly detection. Our research particularly focuses on machine learning-based methods and decentralized detection methods suitable for distributed architectures.

- We summarize the machine learning-based HPC system anomaly detection framework. We compare these methods from multiple dimensions, evaluating them from the perspective of practical application. This can assist researchers in selecting the most appropriate method for specific problems.
- We investigate and describe several important development trends in the field of HPC system anomaly detection.

1.2 Related work

In the research field of anomaly detection in HPC systems, numerous related works exist. Jauk et al. (2019) analyzed and compared over 30 different HPC system failure prediction methods, covering a variety of techniques. Their survey validated that machine learning methods have been widely adopted and demonstrated strong performance in handling complex failure patterns and large-scale data. Canal et al. (2020) provided a detailed discussion on fault detection and recovery techniques ranging from electronic components to system-level in HPC systems. Regarding specific methodologies, Jia et al. (2020) thoroughly explored system log-based methods for diagnosing and detecting system anomalies, while Zhong et al. (2023) offered a comprehensive overview of time-series anomaly detection in Artificial Intelligence for IT Operations. Pang et al. (2021) provided an in-depth analysis of deep learning-based anomaly detection methods, delivering critical insights into this domain. However, despite the widespread application of machine learning in HPC anomaly detection. Recent studies have yet to classify and conduct in-depth analyses of the latest machine learning methods. Therefore, we classify and summarize existing studies systematically and compare different methods in order to better guide users to understand and apply them. Our research focuses on anomaly detection methods based on deep learning and distributed architectures. This not only addresses the gaps in existing surveys within this field, but also aids researchers in better optimizing the performance of detection models in HPC systems.

1.3 Organization

The remainder of this survey is structured as follows: In Sect. 2, we present the definition and background of anomaly detection in HPC systems. In Sect. 3, we outline the challenges associated with machine learning-based anomaly detection methods for HPC systems. Section 4 provides a comprehensive analysis of classical machine learning-based anomaly detection methods. In Sect. 5, we analyze deep learning-based anomaly detection methods. Section 6

discusses several important development trends in this field. Finally, in Sect. 7, we conclude this survey.

2 HPC system anomaly detection

This section outlines the background and framework for anomaly detection in HPC systems. Firstly, we provide a detailed overview of anomaly types in HPC systems. Secondly, we analyze the general anomaly detection framework and its functional components. This establishes a unified terminology for subsequent analysis and discussion of different anomaly detection methods.

2.1 HPC system

An HPC system is a cluster of multiple computers or nodes that communicate and collaborate through high-speed networks to accomplish shared tasks. An HPC system can be broadly divided into three layers: the application support layer, system management layer, and hardware system layer, as illustrated in Fig. 1. The application support layer provides support for user-submitted applications or jobs. The system management layer handles job scheduling and resource management, including various operating systems and file systems. The hardware system layer offers foundational hardware support, comprising computing and storage units as well as high-speed interconnect networks.

2.2 HPC system anomaly

We define anomalies in HPC systems as events or states that exhibit significant deviations from normal operational expectations during system operation. These anomalies are typically caused by abnormal functioning or performance bottlenecks in the components of the HPC system.

HPC anomalies are complex, dynamic, and uncertain, making it hard to define them with a unified standard. Our research focuses primarily on common anomalies that critically impact system performance and security, which mainly include the following categories:

2.2.1 Hardware anomalies

Hardware anomalies, also referred to as failures, are events that occur when hardware components of a HPC system cease to provide correct services (Avižienis et al. 2004). In HPC systems, there exist tens of thousands of hardware units. These components may enter states such as overheating protection, performance degradation, or even complete failure when subjected to disturbances like temperature fluctuations, power supply issues, aging, or human-induced damage.

Hardware anomalies can fundamentally undermine the stable operation of HPC systems. To address this, monitoring and early warning systems, along with disaster recovery and backup systems, are deployed on HPC systems to promptly detect and handle abnormal devices. Meanwhile, as individual hardware anomalies are relatively straightforward to identify. Current research focuses more on emerging

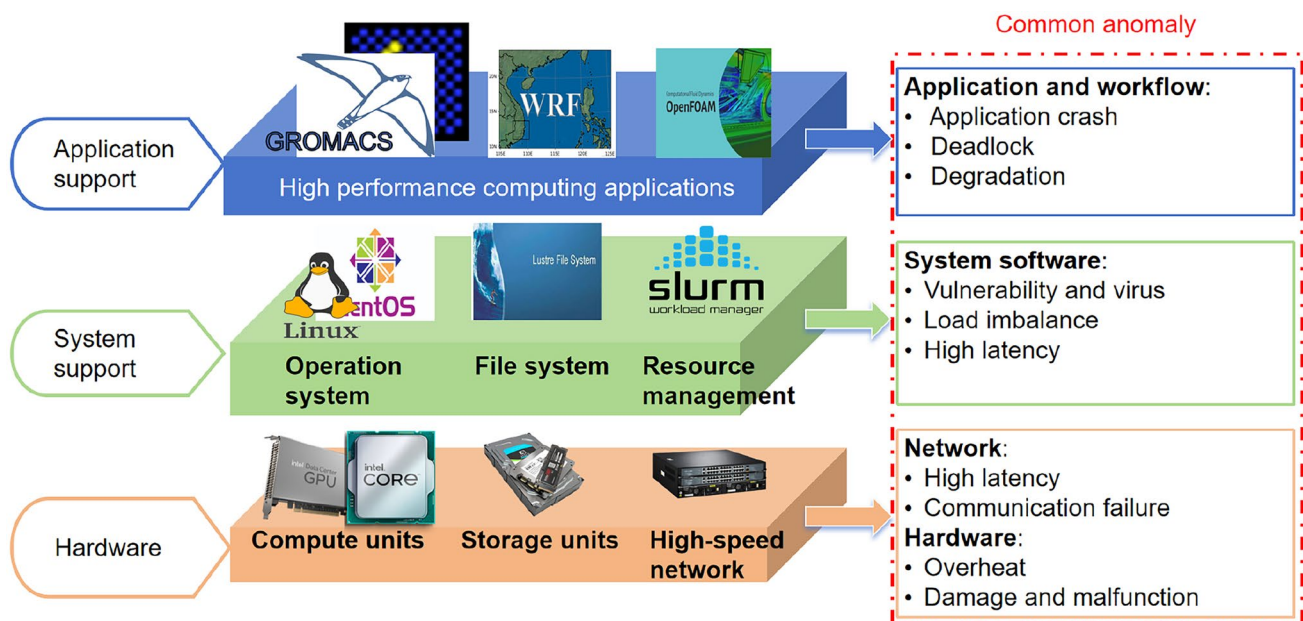


Fig. 1 Hierarchical structure diagram of HPC system and the corresponding anomaly types

hardware such as TPUs (Zu et al. 2024), GPUs, SSDs, and node-level anomalies.

2.2.2 Network anomalies

Network anomalies refer to abnormal behaviors of network equipment or suspicious network states (Zhang et al. 2017). These anomalies primarily manifest as severe communication delays, packet loss, and communication failures. Common causes of network anomalies in HPC systems typically include network device failures, interconnect failures, and traffic congestion (Kumar et al. 2018).

As modern HPC systems' performance heavily relies on high-speed interconnection networks. Network anomalies may lead to cluster performance degradation, task interruptions, and even system crashes (Zhang et al. 2020). Currently, HPC systems are equipped with tools for network health monitoring and bandwidth analysis. Recent research has extensively explored machine learning methods such as LSTM for network traffic prediction and reinforcement learning for intelligent routing optimization. Additionally, we observe that in some networked HPC systems, intrusion detection has emerged as a critical focus area for addressing network anomalies.

2.2.3 Software anomalies

Software anomalies refer to events in HPC systems where running software or applications experience program crashes, performance anomalies (Ibidunmoye et al. 2015), incorrect output results, and other similar issues. These problems are prevalent in scientific applications and certain system software. The causes of such anomalies include configuration errors, programming errors, resource contention, and noise.

Software anomaly detection is a critical yet challenging task in HPC systems. On one hand, administrators and users aim to promptly identify performance bottlenecks and errors in applications. This identification aims to enhance job execution speed and success rates, while concurrently mitigating resource contention and consumption. However, anomaly detection for software and applications faces several challenges, such as interference in data collection, ambiguous definitions of anomalous states, and the lack of access to user information and application code. Currently, a significant number of studies focus on HPC applications like MPI jobs and workflows. They used deep learning methods to uncover complex patterns from their historical data, and have achieved certain levels of success.

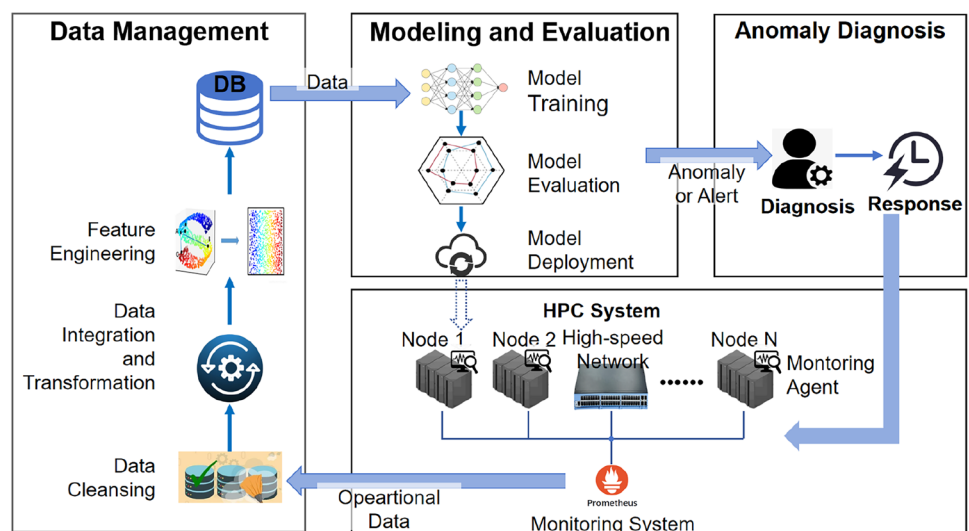
2.3 HPC system anomaly detection framework

HPC system anomaly detection refers to the process of identifying anomalies from the data provided by the HPC system through a series of methods. The goal of anomaly detection is to promptly identify potential issues, prevent failures, and ensure system stability and reliability. Currently, anomaly detection frameworks for HPC systems can generally be divided into four stages: operational data collection, operational data management, modeling and evaluation, diagnosis and anomaly handling. Figure 2 shows the framework diagram of anomaly detection in the HPC system.

2.3.1 Operational data collection

This stage involves gathering monitoring information from various nodes during HPC system operation. Monitoring agents on each node collect monitoring data from multiple layers, including physical hosts, virtual machines, and applications. The acquired data is then transmitted via the network to operational data management. Currently, research

Fig. 2 Framework diagram of anomaly detection in HPC system



on HPC system anomaly detection primarily utilizes three data sources: system log, telemetry data, and software code. These data can be rapidly collected and integrated through monitoring software such as Ganglia and Prometheus. Table 1 provides comparative analyses of the characteristics of the three data source.

2.3.2 Operational data management

This phase focuses on preprocessing collected operational data. Based on the requirements of the detection model, the data undergoes cleaning (handling missing values, invalid entries, and smoothing), integration (merging multi-node monitoring data), transformation, and filtering. Since HPC systems generate vast amounts of high-dimensional data per second. Using raw data for training increases computational time while compromising accuracy and generalization capabilities. Consequently, feature extraction has become a pivotal role in preprocessing. Widely adopted feature extraction methods include the following three categories:

- *Expert experience-based feature extraction.* Combines system characteristics and domain expertise to select features. Rarely used independently in current research, it is often applied as a supplementary method.
- *Statistics-based feature extraction.* Uses correlation analysis, statistical tests, and similar techniques to rapidly identify highly correlated features. However, it struggles with high-dimensional nonlinear feature spaces and requires manual threshold configuration.

Table 1 Data sources for anomaly detection

Data source	Content	Format	Detection target	Time granularity	Limitation
Telemetry data	Hardware and system level metrics	Structured	Hardware status and resources	Millisecond	High dimensionality and significant noise
System log	Running state and operation record	Unstructured	Software logic and process	Minute	High template parsing cost and large storage consumption
Code	Debugging information and static code analysis	Semi structured	Program semantics and calculation errors	Second	Difficult to obtain and analyze

- *Machine learning-based feature extraction.* Employs methods like Principal Component Analysis (PCA), Random Forest, and autoencoder to extract critical features through complex computations. Despite higher computational costs, these methods effectively handle high-dimensional, complex feature spaces and require minimal parameter tuning.

2.3.3 Modeling and evaluation

Designed to predict the likelihood of system anomalies based on input monitoring data and analyze current operational status. During the training phase, preprocessed data is used to train the system status model, typically executed offline to minimize interference with other tasks on the system. In the deployment and operational phase, the model detects potential system anomalies using real-time data. If an anomaly is identified, alert information is reported to the fault diagnosis module for further analysis. From the perspective of historical development, HPC system anomaly detection models mainly consist of the following types:

1. *Rule-based detection.* These methods use manually defined rules and thresholds. System states or performance metrics exceeding or falling below these thresholds flag anomalies.
2. *Classical machine learning-based detection.* These methods train models on historical data using classical machine learning algorithms to rapidly build anomaly detection models. These models are then applied to real-time data for prediction and analysis.
3. *Deep learning-based detection.* These methods leverage deep learning models to perform in-depth analysis of monitoring data. They extract complex features and patterns for more accurate anomaly detection.

Model evaluation is a crucial tool for determining if a model meets system requirements. It serves as a key method to assess the performance and reliability of different models. Table 2 displays common evaluation methods for HPC system anomaly detection models.

2.3.4 Anomaly diagnosis and handling

Anomaly diagnosis is used to analyze and diagnose detected anomalies, identifying their locations and root causes. By examining resource usage and abnormal status alerts in system logs. Operators can pinpoint system metrics and devices with anomalies. However, in complex scenarios—particularly network and application anomalies—diagnosis requires further inference of propagation paths and causal relationships. Currently, effective diagnostic methods include fault

Table 2 An overview of evaluation methods for anomaly detection model

Evaluation method		Evaluation content	Common metrics
Detection	performance	Ability to correctly identify or classify anomalies	F1 score, accuracy, precision, recall
Scalability		Ability in dealing with increasingly large and high dimensional data	/
Robustness		Ability to maintain stability in various uncertainties, perturbations situations	/
Model	performance	The resources and time consumed by model training and running	Training time, inference time, memory usage, CPU usage
Generalization		Ability to maintain stability on new, unseen or unlearned data	/
Explainability		Ability to explain its decisions	/

chain analysis and knowledge graphs. Once the anomaly location and cause are determined, operators can implement targeted solutions. Technical means include replacing faulty components, or terminating abnormal tasks prematurely to release and reclaim resources.

3 Challenge

This section mainly describes the four major challenges faced when running anomaly detection on HPC systems. We investigate the causes, impacts and the latest solutions of these challenges.

3.1 The sample imbalance in anomaly detection

Although the complex structure of systems leads to a variety of anomalies. Even in large systems such as HPC and cloud, anomalies are still very rare compared to the number of normal samples. At the same time, labeling a large amount of data requires a significant investment of time and human resources. This cost further exacerbates the imbalance of anomaly samples.

The lack of anomaly samples has many negative impacts, especially for some supervised learning and classifiers. This is because they require a considerable amount of labeled data to learn decision boundaries. At the same time, constructing

a test set to evaluate model performance also requires a certain number of anomaly samples.

Regarding the issue of imbalanced datasets with anomaly samples, existing research has proposed the following three solutions. The earliest solution involved using simple unsupervised learning methods, such as clustering, to isolate anomalies from a large amount of unlabeled data. However, this method is constrained by noise and computational complexity in high-dimensional spaces. Thus, it limits its scalability and detection performance. The second solution involves using generative models such as autoencoders and generative adversarial networks to detect anomalies. These deep learning methods are more accurate and require only normal samples or very few anomaly samples for training. However, these methods require high-quality datasets and can be challenging to train for some models. The final solution involves artificially generating a sufficient number of anomaly samples using anomaly generation tools like FINJ (Netti et al. 2019) and HPAS (Ates et al. 2019). These tools have been extensively tested on HPC systems and has proven effective. However, these tools cannot cover all types of anomalies, and there remains a gap compared to anomalies in real-world environments.

3.2 High false positive rate in anomaly detection

The false positive rate (FPR) refers to the proportion of normal samples that are incorrectly classified as abnormal samples. In anomaly detection, there are many factors that can affect the FPR. For example, in supervised learning, using a training set with incorrectly labeled data, or setting inappropriate thresholds in unsupervised learning. Both of these can lead to an increase in the false positive rate (Campos et al. 2016). In particular, most existing studies focus only on the detection performance, thereby neglecting the impact of the FPR (Frank et al. 2019).

Frequent false alarms can trigger system checkpoints, causing the system to automatically enter unnecessary backup and recovery states. This misoperation will ultimately waste a large amount of resources and degrade system performance.

To effectively alleviate the FPR, existing methods mainly have the following three improvements. Firstly, semi-supervised learning is used to replace unsupervised learning. That is, by adding a moderate amount of labeled samples during the training process to reduce FPR. Secondly, the quality of training data is improved. By integrating more comprehensive system information and refining feature extraction techniques, more precise features can be extracted. Thirdly, ensemble learning and multi-model fusion are used. They combine unsupervised models such as Isolation Forest,

Autoencoders with supervised models to reduce the misjudgments of individual models by voting mechanism.

3.3 The robustness of anomaly detection

In HPC environments, the impact of noise and some uncertainties is unavoidable, such as operating system noise, network fluctuations, and system mandatory updates (Chunduri et al. 2017). When the system configuration or application changes. The normal and abnormal samples in the new environment often exhibit many new features that the original model has not learned, which makes it difficult for the existing model to effectively determine whether the new samples are abnormal.

When the robustness is low, models are prone to being deceived by adversarial examples, resulting in incorrect outcomes. This leads to a decline in the stability and detection performance of the models. On the other hand, the operational environment of HPC systems is prone to change, making it difficult for old models to function properly in new environments. This necessitates retraining models for the new environment, resulting in additional resource overhead.

There are several methods to enhance the robustness of anomaly detection models. One is to use deep learning methods. Since deep learning neural networks can learn higher-dimensional features. They are capable of effectively reasoning about new and unseen data. Several studies (Aksar et al. 2023, 2024) evaluate model robustness by incorporating unlearned data into the test set. Their experiments demonstrated the reasoning capabilities of deep learning models in extreme and novel environments. Another method is to design a detection framework with self-updating and self-learning capabilities (Farooq and Borghesi 2023). The key of these methods is that the model re-trains based on the latest samples under appropriate conditions. These methods include incremental learning, online learning, and transfer learning.

3.4 The explainability of anomaly detection

Li et al. (2023b) define explainable anomaly detection as a detection method that can provide clear and understandable explanations for the output results. However, for deep learning, the explainability of anomalies is constrained by the highly nonlinear structure and black-box nature of neural networks. So it is unable to possess the traceability and explainability like classical machine learning methods.

Explainability is a very important feature for anomaly detection model. Firstly, it provides developers with an observation window to understand whether the model has learned the correct sample distribution and features.

Secondly, it offers crucial reference for operators to comprehend the causes of anomalies. That helps operators precisely locate the positions of anomalies, saving time on manual diagnosis.

To enhance the explainability of anomaly detection, existing research has approached this from two aspects. The first aspect is improving and simplifying the model. By integrating expert experience to analyze and establish correlations between abnormal system states and observed metrics. Methods like knowledge graphs (Wei et al. 2023), decision trees (Aksar et al. Aksar et al. (2021b)), and fault chain analysis (Molan et al. Molan et al. (2023)) are commonly employed. These approaches can be combined with deep learning-based detection methods to enhance model explainability. The second aspect is developing external explainability tools such as SHAP (Lundberg and Lee 2017) and LIME (Ribeiro et al. 2016) for explaining the discrimination results of neural networks. For example, Aksar et al. (2023) applied the latest anomaly explainability tool CoMTE (Ates et al. 2021) to explain the detection results of autoencoders. Their results demonstrate the effectiveness of this tool.

3.5 Anomaly detection methods based on classical machine learning

In this section, we mainly discuss the HPC system anomaly detection methods based on classical machine learning. Classical machine learning methods, such as statistical and probabilistic analysis, clustering, and decision trees, still possess advantages in terms of resource consumption, explainability, and discrimination speed compared to deep learning methods.

3.6 Anomaly detection methods based on statistic and probabilistic analysis

Methods based on statistic and probabilistic analysis are essentially are processes that involves constructing mathematical probability models to represent the distribution of normal samples. These models identify data points that do not conform to this distribution as anomalies. Common probabilistic models include Bayesian network, Markov model, and logistic regression.

Bayesian networks utilize directed acyclic graphs (DAGs) to represent dependency relationships between variables. They are particularly suitable for modeling causal relationships between system states and monitoring variables. In this methodology, selected monitoring variables are represented as nodes in the Bayesian network, with directed edges indicating dependencies between variables. The system state being monitored is modeled as the sole parent

node of all other nodes in the network. Bayesian inference is then employed to calculate posterior probability distributions of variables. This methodology offers advantages in both training speed and detection efficiency, while enabling quantitative evaluation of each monitoring variable's impact on abnormal states. Molan et al. (2021) implemented a Bayesian online learning-based fault detection method for HPC systems. The key of this method is a Bayesian classifier where each feature corresponds to a system metric characterized by a Gaussian distribution. Through real-time adversarial training, the classifier dynamically updates feature weights, thereby achieving enhanced robustness and explainability. Bayesian networks facilitate anomaly identification and explainability by explicitly modeling causal relationships between system states and monitoring variables. However, the detection performance of such methods heavily relies on assumptions of feature independence and prior knowledge.

Markov chains provide a probabilistic framework for simulating state transitions in systems. These methods employ Markov chains to model the steady state distribution of system log sequences, CPU and memory data (Gu and Wang 2009). Some studies utilize enhanced Markov models such as dual dependent Markov chains (Tan et al. 2012) and Hidden Markov Models (Agrawal et al. 2015) to improve adaptability to real production environments. Haque et al. (2017) used Markov chains to model system logs for detecting node anomalies in HPC. Their method first treated system logs as a set of message sequences and used the algorithm proposed by Teodorescu (Kirkby et al. 2022) to construct the transition matrix. Subsequently, each transition matrix is used to synthesize an expected system log sequence describing normal behavior through random walks. The trained model estimates the occurrence probability of newly observed system logs, identifying messages with the lowest probabilities as anomalies. These studies demonstrate that Markov models can effectively simulate various time-series data in HPC systems and provide reliable prediction data for other classifiers. However, the sequence simulation capability of Markov models is constrained by

training data quality and the inherent memoryless assumption. Compared to LSTM and Transformer, this simulation capacity remains limited.

Logistic regression is a statistical technique for binary classification that establishes a linear relationship between system states and metrics. It employs the sigmoid function to transform linear predictions into probability values between 0 and 1. Alharthi et al. (2021) utilized the Stochastic Gradient Descent Logistic Regression (SGDLR) to learn from labeled log message templates and construct a sentiment lexicon. Leveraging this sentiment lexicon, they assigned sentiment polarity scores to unlabeled log messages. Then, the model determined whether messages indicated errors based on predefined thresholds. However, the linear relationship assumption between metrics and system states in logistic regression may not hold true for all metrics.

3.6.1 Summary

Anomaly detection methods based on statistic and probabilistic analysis offer advantages such as simple implementation, solid theoretical foundations, and strong explainability. Existing research indicates that these methods are primarily used to analyze causal relationships between system states and key variables in telemetry data or system log. However, such methods heavily depend on data distribution and are significantly influenced by environmental noise. They also have stringent application conditions, making them suitable only for anomaly detection in simple scenarios. Therefore, we recommend their use only when the sample size is limited, labeled data is sufficient, and there are clear causal relationships between system states and a small number of indicators. Under these conditions, the method achieves the lowest implementation cost while delivering optimal detection performance and explainability.

3.7 Anomaly detection methods based on clustering

Clustering is an unsupervised machine learning method that groups normal samples together while isolating anomaly samples. In HPC systems, clustering has two key advantages: it does not require labeled samples, and it can be adapted to different detection scenarios by modifying the similarity measure. Clustering methods commonly used for anomaly detection include K-Means, DBSCAN, and fuzzy clustering. Figure 3 shows the classification results of three clustering methods on the same data set.

K-Means (MacQueen 1967) is a typical distance-based partitioning clustering algorithm. Dani et al. (2017) first converted logs from HPC systems into term matrices to extract the frequency of terms. Then they calculated the

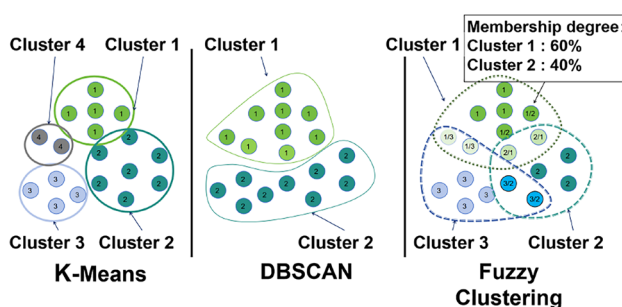


Fig. 3 Classification results of different clustering methods

Euclidean distance between each matrix. After that, they assigned nodes corresponding to each matrix to clusters with the highest cosine similarity metrics, thereby isolating nodes with abnormal status. Other studies employed K-Means to identify normal applications with similar characteristics by analyzing time-series data of resource usage in systems. For instance, Enes et al. (2022) combined resource monitoring metrics from each node in HPC into multivariate time series. They used a pipeline framework incorporating feature extraction and dimensionality reduction to detect application anomalies. After K-Means clustering, normal applications with highly similar time-series features cluster together, while anomalous applications remain distant from these clusters.

Distance-based algorithms are straightforward to implement and computationally efficient, yet they face limitations such as handling only convex-shaped clusters and sensitivity to noise. This stems from their tendency to partition data points into convex regions defined by the nearest cluster centers, making them ineffective for non-convex cluster patterns. Density-based clustering methods can effectively overcome these limitations. DBSCAN serves as a representative density-based clustering algorithm. DBSCAN defines clusters as the largest set of density-connected points, thus enabling the identification of sufficiently dense regions as clusters. For instance, Morrow et al. (2016) used DBSCAN to cluster HPC sensor data for node failure detection. Their experiments compared DBSCAN with K-Means and Gaussian kernel density estimation. The results demonstrated DBSCAN's effective clustering of telemetry data from Cray supercomputers while avoiding the false positive issues observed in the other two methods. Stehle et al. (2024) proposed a method combining DBSCAN clustering with deep learning, which effectively detects anomalies in HPC systems with highly variable configurations, further validating DBSCAN's superior robustness.

However, hard clustering methods such as K-Means and DBSCAN require each sample point to be assigned to only one cluster, which leads to insufficient robustness when processing large-scale data on HPC systems. Therefore, recent studies have employed fuzzy clustering (soft clustering) to enhance the effectiveness of clustering algorithms in HPC environments. Fuzzy clustering allows each sample to belong to multiple cluster centers. There is a membership degree indicating the probability of the sample belonging to each corresponding cluster. This enables fuzzy clustering to handle ambiguous, nonlinear, and noisy data in HPC systems more effectively. El Motaki et al. (2019) utilized the Gath-Geva clustering algorithm to cluster HPC applications based on time series similarity. As a classical fuzzy clustering algorithm, Gath-Geva can automatically determine the optimal number of clusters and exhibits high

stability, making it widely applicable in scenarios such as HPC anomaly detection. However, fuzzy clustering algorithms need to consider the membership degrees of data points to multiple cluster centers, resulting in more complex computational processes. Consequently, algorithm efficiency becomes a critical issue when processing large-scale datasets. To address this, El Motaki et al. (2023) proposed an improved fuzzy clustering algorithm called Fuzzy-PDM for fault detection in HPC systems. This method employed a probability distance measure of object-to-group to enhance the fuzzy maximum likelihood estimation algorithm and improve efficiency. These methods demonstrate that fuzzy clustering enhances clustering robustness and the ability to handle complex data at the cost of computational efficiency and explainability.

Distributed clustering (Rosato et al. 2016) partitions datasets into multiple subsets and processes these subsets in parallel across multiple computing nodes. These distributed architectures offer advantages such as high computational efficiency and strong scalability. However, implementing this method requires addressing challenges like communication overhead and load balancing, which increase the complexity of framework design. Pelaez et al. (2014) employed Distributed Online Clustering (DOC) to detect node failures in HPC systems. Their method partitions the information space of target data and sequentially assigns these regions to nodes for clustering and anomaly detection. Experiments demonstrated that DOC incurs approximately 2% performance overhead while achieving scalability to 5,000 nodes.

3.7.1 Summary

Clustering is an unsupervised learning method, which makes it less affected by sample imbalance. However, it is more sensitive to noise, initial parameters, and feature space dimensionality. The latest research indicates that compared to directly detecting anomalies using clustering, researchers prefer employing clustering for preliminary classification of datasets. This allows them to train models on smaller clusters to enhance model accuracy. Therefore, in simple environments where data distribution is unknown, labeled samples are scarce, and the system requires both real-time performance and explainability. We recommend using clustering to leverage their advantage in rapidly identifying anomaly points. Additionally, this method allows users to adjust clustering algorithms and training parameters based on specific scenarios to meet detection requirements, offering strong flexibility.

3.8 Anomaly detection methods based on decision tree and forest

Decision tree and forest-based anomaly detection methods are a category of supervised learning algorithms. They identify outliers by constructing decision trees or similar structures. These methods typically assume that anomalies exhibit distinct differences from normal instances in feature space, making them easier to be "isolated". During the construction of decision trees, selected system metrics are used to partition the dataset. This process is performed recursively until stopping conditions are met. The tree structure is easy to construct and offers good visualization and explainability. With sufficient labeled samples, these methods demonstrate high detection performance and robustness. Recent studies show their significant role in anomaly detection tasks for HPC. Since they neither require assumptions about data distributions nor frequent computation of similarity metrics, unlike probability-based and clustering-based methods. These methods also show stronger adaptability to noisy high-dimensional data. Notably, the incorporation of various optimization techniques has empowered them to efficiently execute anomaly detection tasks in HPC systems.

Random Forest (RF) (Breiman and Statistics 2001) is a widely used decision tree-based ensemble learning method. Random Forest identifies outliers by constructing multiple decision trees and combining their predictions. This ensemble approach reduces the false positive rate while enhancing the algorithm's robustness. Tuncer et al. (2017, 2019) proposed and improved an HPC application anomaly detection framework based on random forests. Experimental results demonstrate that Random Forest significantly outperformed other baselines in testing. Random Forest achieved overall F1 scores between 97% and 100% with a false positive rate of only 8%. Similarly, Sun et al. (2020) developed a method for modeling and detecting the performance of parallel programs (MPI) with different inputs. This method used Random Forest to automatically analyze and fit the runtime characteristics of MPI programs, which can determine whether anomalies occur during new program execution. Additionally, the approach employs transfer learning to adapt the detection tasks across different platforms.

To further shorten the construction time of decision trees, both XGBoost and Light Gradient Boosting Machine (LGBM) (Ke et al. 2017) utilize the gradient boosting algorithm to optimize the training process. They enhance the memory efficiency of the model and employ GPU acceleration for parallel computation. Such design allows XGBoost and LGBM to fully leverage the advantages of parallel computing on HPC, further speeding up the training process. For instance, Aksar et al. (2021a) first implemented an end-to-end application anomaly diagnosis framework on

HPC systems. They used the monitoring data of applications to train XGBoost and LGBM models separately. Park et al. (2023b) used the scheduler logs on HPC to perform anomaly detection on applications. They conduct feature extraction and selection based on the "evidence weight" and "information value". Their experimental results collectively indicate that XGBoost and LGBM have higher F1 scores than Random Forest. Besides, XGBoost and LGBM demonstrate better performance in terms of false omission rate, false alarm rate, and robustness. However, XGBoost and LGBM require substantial computational resources and a complex parameter tuning process.

Gradient boosting can accelerate the construction of decision trees, but methods based on decision trees and forests still require a large number of labeled samples for training. Constrained by the problem of sample imbalance on HPC systems, obtaining a large number of labeled samples is difficult. To address this issue, existing research has taken two approaches. The first is to reduce the need for labeled samples through active learning. The second is to identify anomalies from unlabeled samples using unsupervised learning methods such as Isolation Forest.

Active learning (Cacciarelli and Kulahci 2024) is a unique semi-supervised learning paradigm that aims to select the most informative samples from a dataset for labeling. Then it retrains the selected model using the newly learned samples during the querying process to minimize labeling costs. Aksar et al. (2022, 2024) designed an HPC application anomaly detection framework that combines active learning with LGBM. Experiments showed that this framework achieved an F1 score of nearly 95%, while the number of labeled samples used was only 1/28 of the supervised learning baseline. However, the selection of the strategy in active learning and the setting of termination conditions is an open question. It may need manual adjustment according to the production environment.

From the perspective of hyperspace, the construction process of the Isolation Forest (iForest) involves continuously splitting sample space with randomly selected hyperplanes. This process will be repeated continuously until all sample points are isolated. This process is entirely based on the distribution of the data itself and does not rely on any external label information. The unsupervised nature and shorter training time of the Isolation Forest make it suitable for HPC environments where data patterns frequently change (Kardani-Moghaddam et al. 2019). Wei et al. (2023) used the Isolation Forest for anomaly detection of application features in HPC jobs. Through feature analysis and the construction of root cause graphs, they are able to integrate all factors affecting job execution and resource usage and display the causal relationships. However, the random feature selection process in Isolation Forest may randomly

neglect some important features, thereby reducing detection performance and efficiency. To address this limitation, studies like (Xu et al. 2023; Zou et al. 2022) have employed various approaches like feature analysis and data compression. Table 3 compares and analyzes three anomaly detection methods based on decision trees and isolation forests, focusing on their differences in working principles and performance.

3.8.1 Summary

Decision trees and forest-based anomaly detection methods are easy to implement, intuitive, and computationally efficient. The introduction of ensemble learning further reduces the false positive rate and enhances detection performance and robustness. The aforementioned studies indicate that these methods are highly suitable for anomaly detection on telemetry data from HPC systems. Therefore, we recommend prioritizing the use of these methods when dealing with large-scale or high-dimensional data. Especially, in scenarios where labeled samples and computational resources are abundant. Random Forest can fully leverage its advantages of precision, explainability, and strong resistance to interference. Moreover, the parallelization of these algorithms is also easy to implement on HPC systems. In some unlabeled scenarios, Isolation Forest is also an effective and efficient method. However, in addition to requiring a large amount of labeled data, this type of method also has some other drawbacks. For example, the calculation cost rises as the number of decision trees increases, and the detection performance is very sensitive to parameter settings.

3.9 Anomaly detection methods based on deep learning

In this section, we mainly discuss the HPC system anomaly detection methods based on deep learning. The rapid development of HPC systems has brought numerous new challenges and demands to the field of anomaly detection, while also creating fresh opportunities. With the increasing computing power of HPC systems, deep learning methods characterized by stronger learning capabilities and enhanced data processing power are being applied to anomaly detection. These methods demonstrate higher levels of automation, improved accuracy, and greater robustness. Consequently,

supported by powerful computing capabilities, deep learning-based approaches are increasingly becoming a hot research direction in the industry.

3.10 Anomaly detection methods based on LSTM

LSTM (Long Short-Term Memory) (Hochreiter and Schmidhuber 1997) is a special type of Recurrent Neural Network (RNN) capable of learning and memorizing long sequence data. In simple terms, LSTM learns normal sequence data and captures long-term dependencies within. In anomaly detection, the LSTM model reconstructs input sequence data and detects anomalies by comparing the reconstruction error between the reconstructed sequence and the input sequence.

Fault chain refers to a cascade of related faults triggered by an initial failure in a system or network. These events occur in a specific sequence and dependency, ultimately leading to the overall failure of the system or network. LSTM can effectively identify and memorize the series of events causing faults, making it applicable for Fault Chain Analysis (FCA). Ren et al. (2021) investigated LSTM-based fault characterization techniques for Blue Gene/L system logs. By parsing system logs, they trained N-gram event sequence patterns using LSTM and extracted fault chain to identify critical events related to system failures. Results showed that the LSTM model achieved over 96% in accuracy and F1 score. It successfully extracted high-confidence fault chains that revealed correlations between fault events. Das et al. (2018) proposed a novel deep learning method with a three-phase training process to predict node failures in HPC. The triply-trained LSTM model accurately identified log event chains leading to failures. Their method predicted abnormal nodes with an average lead time of 2 min, achieving no less than 85% recall and 83% accuracy. Fault chains fully illustrate fault progression and require fewer training samples, thereby facilitating anomaly explanation and root cause analysis. However, collecting fault chain data remains more challenging as it involves extracting relevant information from massive datasets, consuming significant time and resources.

Table 3 Comparison of anomaly detection methods based on decision trees and forests

Method	Learning type	Detection performance	Training speed	Explainability	Efficiency in high dimensional space
Random forest	Supervised	High	Slow	High	High
LGBM and XGBoost	Supervised	High	Fast	Normal	High
Isolation forest	Unsupervised	Normal	Fast	Weak	Weak

3.10.1 Summary

LSTM-based anomaly detection methods learn only normal sequences, making them less affected by anomaly imbalance. The Recurrent Neural Network architecture effectively processes sequential data and captures long-term dependencies, enhancing the robustness and prediction accuracy of anomaly detection. In HPC systems, system logs are chronologically generated event sequences. Thus, LSTM can be employed to learn and predict contextual patterns in log sequences. Compared to Transformers, LSTM demonstrates superior capability in capturing local temporal patterns within short-to-medium sequences. However, when processing high-dimensional data and long time series, LSTM needs to consume more computational resources and may still face the problem of vanishing gradients. Therefore, we recommend prioritizing LSTM for scenarios involving strong local temporal correlations and shorter sequence lengths.

3.11 Anomaly detection methods based on Transformer

The Transformer (Vaswani et al. 2017) replaces the Recurrent Neural Networks (RNN) in LSTM with its unique self-attention mechanism, enabling parallel processing of sequence data and capturing global information. This architecture makes Transformer more efficient and flexible when handling long sequences, while also capturing more precise contextual information. Moreover, it can be extended and modified to adapt to different anomaly detection tasks, achieving integration with other deep learning methods and natural language processing methods.

The self-attention mechanism endows the Transformer with enhanced sequence modeling and processing capabilities on HPC systems, making it widely applicable for anomaly detection tasks. In system log processing, Huang et al. (2020) first introduced the Transformer for anomaly detection. They employed a hierarchical Transformer to capture semantic, contextual, and parameter value information from logs and utilized attention-based classification to compute anomaly probabilities. Their proposed method was tested on three datasets: HDFS (Xu et al. 2009), BGL (Oliner and Stearley 2007), and OpenStack (Du et al. 2017). Experimental results demonstrated that the Transformer-based detection method outperformed LSTM models such as DeepLog in terms of precision and robustness. Alharthi et al. (2023) applied the Transformer to failure detection tasks using HPC system logs. Their model employed two Transformer networks to predict failure locations and fault lead time, respectively. Experiments also confirmed that the Transformer model consistently achieved higher prediction

accuracy than LSTM-based methods. Meanwhile, Transformer reduced model training time, average detection latency, and long-sequence detection latency by approximately 4–8 times compared to state-of-the-art methods. This highlighted the Transformer's ability to improve fault detection speed and accuracy through self-attention mechanisms and parallelization. Han et al. (2023) proposed LTAnomaly, which utilized semantic information, sequential relationships, and component values for log anomaly detection. The method incorporated a Transformer augmented with LSTM as the final classification model. Experiments were performed on BGL datasets with the F1 score reaching 97.5%. This method enhances dependency on feature information and improves the utilization efficiency of log messages.

In the field of telemetry data detection, Tuli et al. (2022) proposed an anomaly detection model named TranAD. Models designed with Transformer-based architectures can accelerate training processes and effectively handle various anomaly detection scenarios. TranAD employed self-adjustment mechanisms and adversarial training to amplify errors and achieve training stability. The experiments were tested and compared on multiple datasets such as SMD and MSDS. Results showed that TranAD achieved 17% and 11% improvements in F1 scores under full and limited training data, respectively. TranAD could also correctly identify root causes for up to 75% of detected anomalies. Additionally, TranAD's training time was only 1% of other baselines. Mingrui et al. (2023) introduced a dual-Transformer-based anomaly detection method. This model utilized a dual Transformer structure to extract associative features from datasets in parallel. It incorporated an improved adaptive multi-head attention mechanism to infer trends across dimensions in multivariate time series data. Experimental results demonstrated that the BTAD model achieved average AUC and F1 scores of 92% and 95.3% across different datasets. On high-dimensional and large-scale datasets, BTAD exhibited significant performance advantages and outperformed baselines like TranAD in overall performance.

3.11.1 Summary

The Transformer's modular design and self-attention mechanism greatly improve its detection performance and efficiency. Transformers can be widely applied in anomaly detection scenarios based on logs and telemetry data. Compared to LSTM, the Transformer can attend to all positions in a sequence simultaneously through self-attention mechanisms. This makes it better suited for capturing temporal and spatial relationships in long-sequence data. However, the Transformer has limited capability in detecting local anomalies and relies on positional encoding to incorporate location information. Its computational complexity grows

notably with longer sequences. Yet, it supports parallel computation across multiple attention heads, enabling higher efficiency with sufficient computational resources. Therefore, Transformer offers broader applicability, excels in handling complex long-sequence data, and achieves higher computational efficiency.

3.12 Anomaly detection methods based on deep learning and natural language processing

System log-based anomaly detection methods have wide-ranging applications and research (Jia et al. 2020). This is because the fact that the system log has a wide coverage and can support the framework to diagnose anomalies at a more fine-grained level. However, system logs often lack a unified format and contain substantial natural language content. That makes them hard to directly use as input data for training. Therefore, the fundamental framework for log-based anomaly detection typically consists of three components: log parsing, log vectorization, and model training and detection. In previous research, deep learning methods such as LSTM and Transformer have been employed in the detection phase to identify anomalous sequences. These methods have capabilities in learning and memorizing temporal sequences. With advancements in natural language processing (NLP) (Otter et al. 2021), numerous studies now combine deep learning with NLP to uncover deeper patterns and information from system logs. Figure 4 shows the process of the deep-learning-based anomaly detection method for system logs.

In early research, the DeepLog proposed by (Du et al. 2017) utilized sequences of log keys and parameter values extracted from logs to train an LSTM model for system anomaly detection. Deeplog was experimentally validated on Blue Gene/L supercomputer system logs. However, DeepLog only employed log template indices to learn sequential patterns and failed to sufficiently extract semantic information. These shortages lead to a high rate of false positives.

Inspired by DeepLog, semantic extraction from system logs became a key research direction. Meng et al. (2019) developed LogAnomaly, the first anomaly detection framework incorporating log semantic information. They used Template2Vec to extract semantics, enabling the LSTM to better understand semantic relationships between log templates. The LogRobust framework proposed by (Zhang et al. 2019) first converted logs into semantic vectors using FastText and TF-IDF. They employed attention-based Bi-LSTM for anomaly detection to enhance robustness. Li et al. (2024) transformed HPC system logs into log vectors via MiniLMv2 and detected anomalies using an autoencoder. Zhang et al. (2021) applied sentiment analysis (Feldman 2013) from NLP to anomaly detection in parallel file systems (PFS) on HPC. Their method analyzed source code of PFS to collect log statements, which were fed into a Bi-LSTM to train a sentiment model. Anomalies were identified based on the model's sentiment prediction outputs.

Natural language processing offers advantages in semantic extraction, such as fast transformation and low computational complexity. However, challenges remain in handling complex semantic scenarios. To solve the problem of

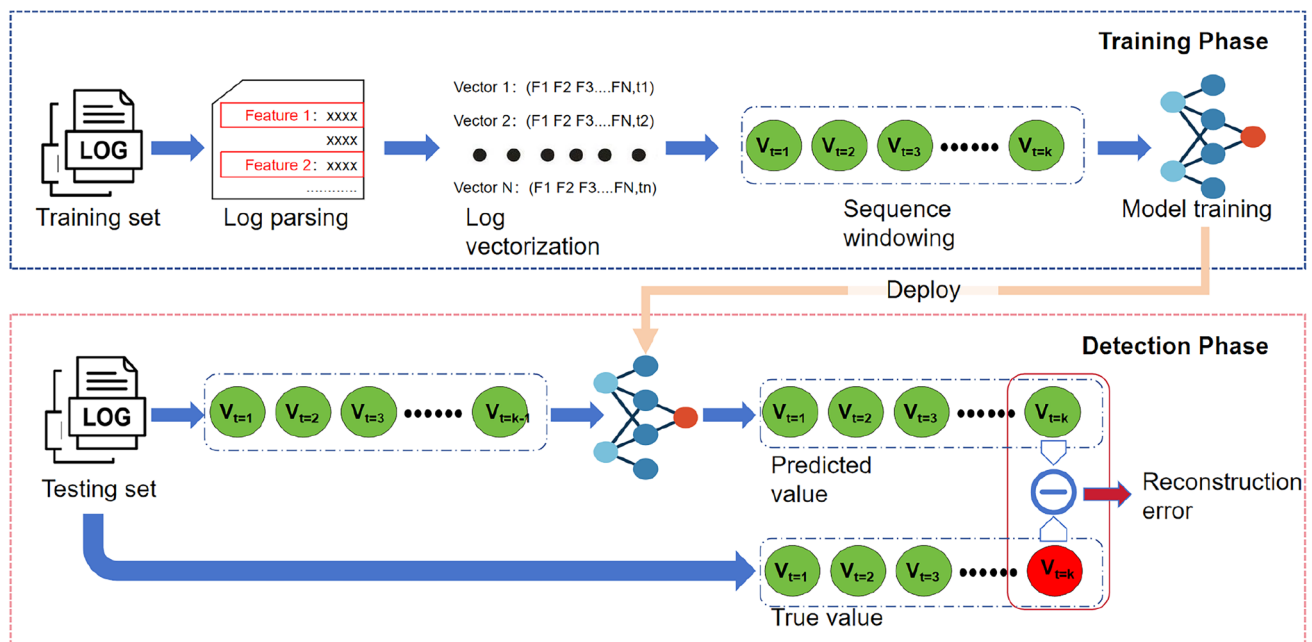


Fig. 4 Flow chart of system log anomaly detection based on deep learning

language extraction in intricate environments and improve the performance of NLP tasks, some preprocessing models based on deep neural networks are used in the latest research. Bidirectional Encoder Representations from Transformers (BERT) (Jacob et al. 2019)—a Transformer-based pretrained NLP model—has played a significant role in recent studies. By leveraging its bidirectional Transformer architecture, BERT enables comprehensive semantic understanding and accelerates model training efficiency.

Li et al. (2023a) employed BERT to encode log templates and used Bi-LSTM for system anomaly detection. Their method combined temporal and semantic embeddings, achieving higher robustness and precision than LogRobust and DeepLog. Fu et al. (2023) utilized a Transformer encoder and a novel text mining technique for semantic extraction. They designed a hybrid model integrating Mogrifier LSTM and CNN for fault detection. AllInfoLog Xiao et al. (2023) further leveraged all log information to detect diverse anomaly types. It employed four RoBERTa-based encoders to separately extract semantic, parametric, temporal, and other features for training a Bi-LSTM model. RoBERTa, an improved variant of BERT, exhibits superior performance. AllInfoLog achieved an F1 score of nearly 99.4% on BGL datasets outperforming all aforementioned methods.

Other methods like logBERT (Guo et al. 2021b), Bertlog (Guo et al. 2021a), LAnoBERT (Lee et al. 2023), and LogFiT (Almodovar et al. 2024) also utilized BERT for log semantic extraction. However, these methods lack scalability due to their custom tokenization, which is incompatible with log systems featuring distinct vocabularies. This limits their applicability to newly deployed log systems. To address this, Jin et al. (2024) explored large language models (LLMs) in computational workflow anomaly detection. They mainly studied two methods, Supervised Fine Tuning (SFT) and In Context Learning (ICL), and evaluated their performance, efficiency and generalization ability. Experiments showed that SFT models generally outperformed pretrained models on test sets. Additionally, SFT combined with transfer learning enhanced LLM performance on new tasks through fine-tuning. Meanwhile, ICL models demonstrated advantages in flexibility and explainability. Since CoT prompts made LLM decision-making more transparent.

3.12.1 Summary

Deep learning-NLP hybrid anomaly detection methods effectively address diverse logs in HPC systems. These methods combine non-structured data processing with precise contextual extraction. Although current methods widely adopt BERT and LLMs to automate semantic extraction, fine-tuning for new log formats remains unavoidable.

Simultaneously, such methods require substantial labeled data and involve training different neural networks for both semantic parsing and model detection. Consequently, their practical application may be constrained by the scarcity of anomaly samples and limitations in computational resources. Therefore, we recommend balancing detection performance and resource consumption when employing this approach. It is advisable to select appropriate log parsing and detection methods based on the complexity of the logs.

3.13 Anomaly detection methods based on autoencoder

An autoencoder (AE) consists of two components: an encoder and a decoder. The encoder compresses input data into a lower-dimensional latent space. The decoder reconstructs data from this latent space representation. Unlike simple memorization of training samples, the encoder learns intrinsic data patterns and essential features. For anomaly detection, autoencoder measures feature similarity between new data and normal patterns in the latent space. Anomalies are identified by calculating reconstruction errors between input samples and their decoded outputs. The key-feature-based reasoning and comparison in autoencoders leads to better generalization and stronger model robustness.

In early research, Borghesi et al. (2018, 2019) explored the use of autoencoders for HPC fault detection. Their experiments demonstrated that autoencoders trained solely on normal samples could achieve performance comparable to state-of-the-art supervised learning methods. Moreover, they could effectively identify novel and unlearned anomalies with limited labeled data. To further investigate autoencoder performance, Aksar et al. (2021b) recently applied autoencoders for HPC application anomaly detection. Their method first employed stacked autoencoders to learn features of normal and anomalous samples. Subsequently, the encoded features are fed into classical machine learning models to train classifiers. This phase used limited labeled samples through supervised learning for anomaly diagnosis. To obtain sufficient anomaly samples, they leveraged HPAS to simulate anomalies. Experimental results show this framework achieved an average F1 score 11% higher than contemporary supervised methods and reduced time and resource costs. In the latest advancement, Aksar et al. (2023) adopted variational autoencoders (VAE) (Kingma and Welling 2013), which exhibit superior and more stable reconstruction capabilities, for unsupervised learning of normal sample. Additionally, they integrated a multivariate time-series counterfactual explanation tool (CoMTE) to explain detection outcomes.

Similar to the Transformer, autoencoders can be combined with other methods to address specific problems. This approach effectively enhances the ability to extract contextual information from time-series data or improves the training efficiency. For example, Borghesi et al. (2022) proposed a fault detection method for HPC systems that combines DNNs with autoencoders. This method introduced supervised learning to mitigate the FPR and scalability issues of autoencoder. Molan et al. (2023) integrated autoencoders with LSTM to identify node failures in HPC systems. This method leveraged the LSTM capability to capture long-term dependencies and improve performance. Catillo et al. (2022) converted logs into numerical scores and used autoencoders for fault detection. On the BGL dataset, this method achieved training time under 1 min and reached a maximum recall rate of 99% and an accuracy rate of 98%. Farzad and Gulliver (2020) proposed an unsupervised log anomaly detection model based on Isolation Forest and dual deep autoencoders. This method improved dataset quality and achieved rapid training on BGL.

At the end of his study (Borghesi et al. 2019), Borghesi explored autoencoders based on a distributed architecture. Experimental results demonstrated that autoencoders trained on a single node achieved higher accuracy with shorter training time. Revealing that such distributed architectures could effectively reduce overall system overhead without compromising performance. Molan et al. (2022) applied autoencoders to extract and compress data from each compute node. They utilized clustering at the control node to identify abnormal nodes from the aggregated compressed data. Farooq and Borghesi (2023) employed federated learning to enhance the performance of HPC anomaly detection models. The method's core involved a central control node applying FedAvg to compute a weighted average of each node's autoencoder weights. Then, the global model was updated and redeployed to other nodes. Experiments showed this framework reduced the average training time to 1.2 weeks, achieving a nearly 15-fold speed improvement. In distributed architectures, autoencoders efficiently and automatically extract data features from nodes to accomplish data compression. This process reduces the size of transmitted data while safeguarding user privacy.

3.13.1 Summary

Autoencoders serve as an effective and practical semi-supervised approach for anomaly detection tasks based on telemetry data. The neural network design of autoencoders enables them to efficiently process high-dimensional data and extract features representing normal patterns. The training process of autoencoders is simple and requires no extensive labeled samples. They also exhibit high robustness and

strong generalization capabilities, making them well-suited for nearly all anomaly detection tasks based on telemetry data. On the other hand, autoencoder still exhibits two main limitations. Firstly, reconstruction errors are susceptible to various factors such as the quality of normal samples and environmental noise. Secondly, the threshold for determining anomalies carries a degree of subjectivity and cannot automatically adapt to practical scenarios.

3.14 Anomaly detection methods based on Generative Adversarial Network

Generative Adversarial Network (GAN) consists of two neural networks: a generator and a discriminator. The generator attempts to create synthetic samples resembling the training data. The discriminator strives to distinguish between real samples and the fake samples generated by the generator. GAN's unique adversarial training approach closely mirrors the anomaly detection process. It eliminates the need for feature selection and manual labeling of samples. Once trained, the generator can produce sufficiently realistic data or generate anomalous data by introducing noise, while the discriminator acquires enhanced capability to identify anomalies.

A major advantage of GAN is their ability to generate a sufficient number of normal or anomalous samples through the generator. This advantage helps alleviate the problem of anomaly imbalance in real-world environments. Farzad and Gulliver (2019) proposed an anomaly detection model that uses Sequence Generative Adversarial Networks (SeqGAN) and oversampling of log messages to handle imbalanced log data. The results indicate that the anomalies generated by GANs in combination with oversampling have better quality. And these generated samples effectively improve various detection performance. Luo et al. (2020) used reinforcement learning to construct a state transition process for anomaly detection in HPC applications. They used a GAN-based method to generate a sufficient amount of training data. Xia et al. (2019, 2021) proposed a system software anomaly detection framework called LogGAN. This method combines LSTM and Generative Adversarial Networks to analyze system logs. Experimental results demonstrated that this method outperforms baselines such as DeepLog and iForest in terms of precision and false alarm rates. Zhao et al. (2022) proposed a GAN based on the Transformer encoder design and named it Trine. This combination can more effectively capture log sequence features and accelerate convergence. Trine achieved higher performance than LogGAN on the OpenStack dataset.

Although the unique adversarial training of GAN can bring many advantages, it requires a large amount of computing resources. Also, the model has a slow and unstable

convergence speed. This problem hinders the application of the model in actual production environments. Especially, in cloud environments, resource competition is more obvious. For this reason, many studies use the latest technical means such as NSGA-III (Yang and Zeng 2022) and BEGAN (Park et al. 2023a) to improve the stability of the GAN training process and reduce the occupied resources.

3.14.1 Summary

It is evident that GAN possesses robust anomaly detection capabilities. Through adversarial learning, they can effectively address challenges such as insufficient or imbalanced

Table 4 Strengths and weaknesses of various methods

Method	Strengths	Weaknesses
Statistic and probabilistic analysis	Easy to implement Low resource consumption High explainability	Difficult to model complex relationships Labeled data is required Low efficiency in high dimensional data
Clustering	No need for labeled data Easy to implement High flexibility	Require human intervention Sensitive to initial parameters and noise High computational complexity
Tree and forest	High calculation speed High detection performance High efficiency in high dimensional data High explainability	Labeled data is required Not applicable to small scale data sets Sensitive to initial parameters
LSTM and Transformer	Suitable for processing system log High detection performance and robustness High sequence prediction ability	Labeled data is required Require log processing Limited explainability
Autoencoder	High detection performance and robustness High generalization ability Hardly any labeled data are required	Manual threshold setting is required Limited explainability Limitations of the reconstruction error hypothesis
GAN	Suitable for telemetry data Hardly any labeled data are required High detection performance and robustness High sample synthesis ability	Hard to train High consumption Mode missing

samples in real-world environments. GAN is applicable for anomaly detection tasks based on logs and telemetry data. However, GAN-based anomaly detection methods face several limitations that restrict their practical applications. Firstly, discrepancies exist between samples generated by the generator and real-world samples, accompanied by issues of mode collapse. Secondly, GAN model training is a resource-intensive and unstable process, since the generator and discriminator can easily fall out of sync during training. Consequently, GAN-based methods are deployed only in scenarios demanding extremely high detection accuracy and scarce labeled data. Given their stringent operational requirements and training complexity. Such methods are primarily used in cloud and certain Internet of Things (IoT) for network and intrusion detection. Research on using this method in HPC environments is still relatively rare. This discrepancy may stem from the critical and complex security demands in cloud and IoT environments. Most HPC systems currently lack such urgent requirements.

4 Discussion

In this work, we review methods that leverage machine learning for anomaly detection in HPC systems. We discuss how these methods address various challenges in HPC systems. Table 4 revisits and compares the strengths and weaknesses of different methods. Through this comprehensive review, we demonstrate the critical importance and significant impact of machine learning methods, while identifying several notable trends.

4.1 Trends in anomaly detection data sources

Anomaly detection methods still predominantly rely on single data sources (logs/telemetry data), but approaches using multi-source heterogeneous data continue to evolve.

Methods utilizing single data sources reduce data collection and processing costs while facilitating comparisons with other baselines. However, the advent of lightweight monitoring tools, enhanced data collection capabilities, and decreased processing as well as storage costs have driven shifts in data sources.

Firstly, systems now provide more granular data, expanding the feature space available to models. This change offer a comprehensive perspective for analyzing spatiotemporal characteristics of system data. For instance, studies like (Aksar et al. 2023, 2021b) leveraged fine-grained telemetry data collected by LDMS. Besides, they used anomaly injection tools like HPAS to supplement missing anomaly data and enhance model robustness and detection performance. Research such as (Molan et al. 2021, 2023) collected labeled

data from monitoring tools like Nagios to train models, providing critical references for system state analysis. These studies demonstrate that finer-grained data improves detection performance. And the development of advanced data collection tools represents a key technological driver for anomaly detection frameworks.

On the other hand, using multi-source heterogeneous data in anomaly detection models has become a promising trend. Recent studies have explored methods combining different data sources and highlighted their advantages. For example, approaches detecting anomalies from both logs and telemetry data (Gurumdimma et al. 2016; Chuah et al. 2013, 2016) can integrate results from both sources to reduce overall false positive rates. Methods combining system logs with source code analysis (Xu et al. 2009; Hickman et al. 2018; Zhang et al. 2023) enable precise mapping of log anomalies to corresponding software code, facilitating application anomaly diagnosis and localization.

These studies reveal that multi-source heterogeneous data can offer a viable solution in previously challenging scenarios. These scenarios include application performance analysis and system fault diagnosis. Future research could further explore unified representation and processing techniques for multi-source heterogeneous data. Researchers can employ semantic alignment methods and tensor fusion models to achieve semantic consistency across data sources and simplify model utilization and processing.

4.2 Trends in anomaly detection feature engineering

Feature engineering techniques have become a key module of anomaly detection frameworks. Feature engineering (extraction and selection) serves as a critical step for processing high-dimensional data and enhancing model performance and generalization capabilities. However, with technological advancements, systems now offer an increasing number of observable metrics and features. This situation drives the evolution of feature engineering toward greater efficiency and automation.

Initially, operations personnel manually extracted and selected important features from datasets based on experience. This manual approach failed to effectively reduce dimensionality and often overlooked significant features. Subsequently, feature extraction method based on statistical characteristics and correlation analysis gained prevalence. They employ methods like statistical feature analysis and correlation analysis to derive high-value features from data. These methods are widely used for feature extraction from logs and time series data. A range of efficient and user-friendly tools have been developed for this purpose, making it a widely accepted method in feature

engineering. Nevertheless, while such feature extraction methods improve accuracy, they still require manual threshold setting.

The latest machine learning-based feature extraction methods, supported by large datasets, employ deep learning algorithms such as autoencoders to automatically and efficiently extract features. These approaches demonstrate superior adaptability to high-dimensional and complex data, and their adoption is growing rapidly. We recommend that future research should optimize feature engineering by balancing comprehensive performance and resource consumption. Firstly, regarding performance enhancement. Further exploration of feature extraction leveraging machine learning and natural language processing is warranted. Particularly for spatiotemporal features such as job parallelism, runtime and I/O patterns in HPC system. Deeper investigation into these characteristics could significantly improve model learning capabilities. Additionally, integrating multiple feature extraction methods could boost accuracy. A representative application involves using machine learning methods to automatically reduce feature space dimensionality. Followed by expert-guided fine-tuning of extracted features. Secondly, we note that in some studies (Tuncer et al. 2019), feature extraction has become the most time-consuming part in frameworks. This observation highlights the critical need to reduce feature engineering time through measures like parallel computing and data compression. These measures can improve model efficiency and real-time performance. Such optimizations will be essential for advancing practical implementations.

4.3 Trends in anomaly detection method

Machine learning has become the mainstream method for anomaly detection. The role of deep learning methods in this field remains to be further explored.

After over two decades of development, machine learning-based methods have gradually achieved dominance in both quantity and proportion. This is attributed to the enhanced computational power of HPC systems and the advantages of machine learning in handling large volumes of complex, high-dimensional data. The advancement of machine learning methods has driven the evolution of anomaly detection. Techniques such as neural networks, adversarial training, and transfer learning in deep learning have effectively addressed challenges faced by traditional methods. These challenges include scarcity of abnormal samples, robustness, scalability and generalization ability.

To overcome the limitations of single machine learning algorithms, many studies have combined multiple machine learning algorithms to improve overall performance. Some methods integrate decision trees with deep learning models,

leveraging decision trees to enhance the explainability of deep learning. Other methods combine multiple deep learning algorithms to achieve more accurate detection results and higher performance.

Therefore, we suggest that future research could continue to explore the potential and application scope of existing deep learning methods. Particularly, the research and applications of the latest deep learning should be emphasized. Approaches like GANs and diffusion models remain lacking in the field of HPC anomaly detection. On the other hand, deeper investigations should address the limitations of current methods in practical applications. For instance, we need to tackle the threshold adaptation challenge of autoencoders in dynamic environments. Effective dynamic threshold mechanisms could be designed to automatically adjust thresholds based on the latest error distributions. Additionally, enhancing models' continuous learning and optimization capabilities would be valuable. When newly detected data distributions deviate significantly from original training data, generative replay and dynamic network expansion techniques could be employed to improve neural networks for automatic environmental adaptation. Regarding training difficulties in methods like GANs, relevant work on HPC platforms could draw insights from cloud platform studies (Yang and Zeng 2022; Park et al. 2023a; Qi et al. 2023) and IoT platform research (Tian et al. 2020; Benaddi et al. 2022). For example, Park et al. (2023a) utilized the state-of-the-art Boundary Equilibrium GAN (BEGAN) (Berthelot et al. 2017) combined with autoencoders for model construction. BEGAN synthesizes more reasonable data by leveraging the Wasserstein distance between reconstruction error distributions. This improvement stems from the Wasserstein distance's inherent property of constructing a smoother loss function better suited for gradient optimization. This property fundamentally mitigates common GAN training issues like gradient oscillation and sudden divergence. Meanwhile, Tian et al. (2020) adopted a distributed architecture that leverages local data to train GAN on individual nodes, effectively reducing both training costs and communication overhead.

4.4 Trends in anomaly detection evaluation metrics

The evaluation system for anomaly detection methods has undergone significant evolution. It has changed from relying on single performance metrics to incorporating multidimensional comprehensive assessments. This trend has been fully corroborated in recent studies. Cutting-edge research not only focuses on detection performance but also organically integrates multidimensional evaluation indicators such as real-time performance, robustness and explainability. These advancements collectively establish a comprehensive

evaluation system, enabling precise characterization and comprehensive analysis of method effectiveness.

In most studies, evaluation systems only include detection performance metrics such as accuracy, F1 score, and recall. While this provides a relatively unified standard for comparing detection performance across different methods, Salfner et al. (2010) and Taerat et al. (2010) have identified critical issues with existing evaluation system. Firstly, these studies predominantly focus on detection performance while neglecting factors like time delays and resource consumption. These factors render the models challenging to deploy in real-world production systems. Secondly, evaluation results are derived from diverse test sets and environments, undermining the comparability of different methods.

We believe that the development of current evaluation metrics is driven by practical engineering demands. Many anomaly detection efforts introduce complex models to achieve extremely high precision, yet such trade-offs require careful consideration in real-world production environments. This is because operational personnel in actual production settings need to comprehensively evaluate multiple metrics beyond just detection performance. Therefore, more research attention should be directed toward practical application-related indicators such as model scalability, robustness, and explainability. Regarding generalizability, there is an urgent need for a unified evaluation system to accurately assess the real-world performance differences between various methods. We observe that existing studies have achieved remarkably high detection performance, with average accuracy, precision, and F1 score approaching 100%. However, significant discrepancies exist across these experiments in terms of datasets, environmental configurations, and other factors. Even when using identical datasets (such as BGL logs) for testing, the evaluation methodologies for temporal performance vary substantially and often lack clarity, making it impossible to effectively compare practical performance and real-time capabilities between existing studies. We propose designing a benchmarking tool that unifies and standardizes the datasets, node counts, and temporal evaluation metrics used by models, enabling better comparison of performance and bottlenecks across different approaches.

4.5 Trends in distributed architectures for anomaly detection

The distributed architecture for anomaly detection is a unique implementation method of anomaly detection models in HPC systems, which can enhance the performance of HPC systems and improve detection efficiency.

The distributed architecture can be applied to various machine learning approaches such as clustering, Random

Table 5 Detection methods using distributed architecture and their performance

Method	References	Principle	Scalability and speedup
Decentralized clustering	Pelaez et al. (2014)	Allocate the data to various nodes for clustering	Handling at least 60k events using up to 5k nodes Less than 10MB of RAM Less than 5MB of bandwidth Around 2% performance overhead
Node-specific model	Borghesi et al. (2019)	Each node uses the data generated by itself to train the model	Increase the training speed by 11 times
Federated learning	Farooq and Borghesi (2023)	Use federated learning to update the global model through the weight of the autoencoder on each node	Increase the data collection speed by 15 times

Forest, autoencoders, and GANs. These studies demonstrate that effective distributed architectures exhibit the following key roles and development trends.

Firstly, data partitioning and deployment based on HPC system architecture. In the studies mentioned above (Pelaez et al. 2014; Borghesi et al. 2019), data is partitioned and distributed across multiple nodes for training and detection. This approach reduces inter-node communication costs while avoiding performance bottlenecks caused by centralized storage and processing on a single node, thereby achieving high scalability. Future research could further optimize data partitioning and deployment strategies by considering system resource configurations or job types. For example, regard the nodes performing similar tasks as a group and select a small number of nodes from it to collect data. Then select adjacent computing nodes to train a detection model for detecting abnormalities in this group. Alternatively, dynamic deployment strategies could be designed to adjust data partitioning granularity and redistribute data from high-load nodes to idle ones based on real-time system load, data distribution, and network status.

Secondly, model parallelization and federated learning based on distributed architectures. Among the methods discussed, methods like Random Forest and Transformer can be accelerated through parallel computing. Typical applications of such methods include XGBoost, LGBM, and the self-attention mechanisms in Transformers. These approaches employ model parallelism to split decision trees or neural network layers. They combine asynchronous updating mechanisms to accelerate training. This enables more efficient utilization of HPC system resources

for enhanced computational efficiency when processing large-scale datasets. In studies such as (Molan et al. 2022; Farooq and Borghesi 2023; Tian et al. 2020; Benaddi et al. 2022), researchers employ federated learning and data compression to improve model detection performance. This approach not only reduces data transmission costs but also protects sensitive information, while enabling models to learn data features across nodes. Consequently, it has been widely adopted in distributed systems like HPC and Cloud.

Table 5 shows the anomaly detection frameworks developed based on the distributed architecture on the HPC system and their effects. We believe model parallelization techniques in anomaly detection could be further integrated with HPC resource management and scheduling modules. By dynamically collecting and analyzing node resource status and network traffic, adaptive data partitioning and model parallelization schemes could be developed. Federated learning, on the other hand, could be combined with feature engineering techniques. Deep learning-based feature extraction and selection methods may enhance the quality of transmitted data, reduce data size, and prevent cross-node transmission of critical or private information.

5 Conclusion

As HPC systems become vital for social development and technological advancement, implementing efficient anomaly detection techniques is essential for their performance and reliability. With increasing complexity, establishing intelligent, lightweight, and user-friendly frameworks for anomaly detection has become a key objective for researchers and operational personnel. This paper summarizes the definitions and challenges of machine learning-based anomaly detection in HPC systems, analyzes existing methods through a multi-dimensional approach, and highlights significant trends to provide insights for future research.

Acknowledgements This work was supported by the National Key Research and Development Program of China (2023YFB3001802).

Author contributions Lu Yiming and Zhang Wei made equal contributions to this work. The corresponding author is Yong Dong.

Data availability Data availability is not applicable to this article as no new dataset were created or analyzed in this study.

Declarations

Conflict of interest The authors have no Conflict of interest to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format,

as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Agrawal, B., Wiktorski, T., Rong, C.: Analyzing and predicting failure in hadoop clusters using distributed hidden markov model. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* **9106**, 232–246 (2015). https://doi.org/10.1007/978-3-319-28430-9_18
- Aksar, B., Schwaller, B., Aaziz, O., Leung, V. J., Brandt, J., Egele, M., Coskun, A. K.: E2ewatch: An end-to-end anomaly diagnosis framework for production hpc systems. In: Leonel Sousa, Nuno Roma, and Pedro Tomás, editors, *Euro-Par 2021: Parallel Processing*, pp. 70–85, Cham, Springer International Publishing. ISBN 978-3-030-85665-6 (2021)
- Aksar, B., Zhang, Y., Ates, E., Schwaller, B., Aaziz, O., Leung, V. J., Brandt, J., Egele, M., Coskun, A. K.: Proctor: A semi-supervised performance anomaly diagnosis framework for production hpc systems. In: Bradford, L., Chamberlain, Ana-Lucia Varbanescu, Hatem Ltaief, Piotr Luszczek. (eds.) *High Performance Computing*, pp. 195–214, Cham, Springer International Publishing. ISBN 978-3-030-78713-4 (2021)
- Aksar, B., Sencan, E., Schwaller, B., Aaziz, O., Leung, V.J., Brandt, J., Kulis, B., Coskun, A.K.: Albadross: Active learning based anomaly diagnosis for production HPC systems. In: 2022 IEEE International Conference on Cluster Computing (CLUSTER), 369–380, (2022). <https://doi.org/10.1109/CLUSTER51413.2022.00048>
- Aksar, B., Sencan, E., Schwaller, B., Aaziz, O., Leung, V. J., Brandt, J., Kulis, B., Egele, M., Coskun, A. K.: Prodigy: Towards unsupervised anomaly detection in production hpc systems. In SC23: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 01–14, (2023). <https://doi.org/10.1145/3581784.3607076>
- Aksar, B., Sencan, E., Schwaller, B., Aaziz, O., Leung, V.J., Brandt, J., Kulis, B., Egele, M., Coskun, A.K.: Runtime performance anomaly diagnosis in production hpc systems using active learning. *IEEE Trans. Parallel Distrib. Syst.* **35**(4), 693–706 (2024). <https://doi.org/10.1109/TPDS.2024.3365462>
- Alharthi, K. A., Jhumka, A., Di, S., Gui, L., Cappello, F., McIntosh-Smith, S.: Time machine: Generative real-time model for failure (and lead time) prediction in hpc systems. In: 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 508–521, (2023). <https://doi.org/10.1109/DSN58367.2023.00054>
- Alharthi, K.A., Jhumka, A., Di, S., Cappello, F., Chuah, E.: Sentiment analysis based error detection for large-scale systems. In: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pages 237–249, (2021). <https://doi.org/10.1109/DSN48987.2021.00037>
- Almodovar, C., Sabrina, F., Karimi, S., Azad, S.: Logfit: Log anomaly detection using fine-tuned language models. *IEEE Trans. Netw. Serv. Manage.* **21**(2), 1715–1723 (2024). <https://doi.org/10.1109/TNSM.2024.3358730>
- Ates, E., Zhang, Y., Aksar, B., Brandt, J., Leung, V.J., Egele, M., Coskun, A.K.: Hpas: An hpc performance anomaly suite for reproducing performance variations. In: *Proceedings of the 48th International Conference on Parallel Processing, ICPP '19*, New York, NY, USA. Association for Computing Machinery. ISBN 9781450362955. (2019) <https://doi.org/10.1145/3337821.3337907>
- Ates, E., Aksar, B., Leung, V.J., Coskun, A.K.: Counterfactual explanations for multivariate time series. In: 2021 International Conference on Applied Artificial Intelligence (ICAPAI), pp. 1–8, (2021). <https://doi.org/10.1109/ICAPAI49758.2021.9462056>
- Avizienis, A., Laprie, J.-C., Randell, B., Landwehr, C.: Basic concepts and taxonomy of dependable and secure computing. *IEEE Trans. Dependable Secure Comput.* **1**(1), 11–33 (2004). <https://doi.org/10.1109/TDSC.2004.2>
- Benaddi, H., Jouhari, M., Ibrahim, K., Othman, J.B., Amhoud, E.M.: Anomaly detection in industrial iot using distributional reinforcement learning and generative adversarial networks. *Sensors*, **22**(21), (2022). <https://doi.org/10.3390/s22218085>
- Berthelot, D., Schumm, T., Metz, L.: Began: Boundary equilibrium generative adversarial networks. *Statistics*, (2017)
- Borghesi, A., Bartolini, A., Lombardi, M., Milano, M., Benini, L.: Anomaly detection using autoencoders in high performance computing systems. *ArXiv*, abs/1811.05269, (2018). <https://api.semanticscholar.org/CorpusID:53288981>
- Borghesi, A., Bartolini, A., Lombardi, M., Milano, M., Benini, L.: A semisupervised autoencoder-based approach for anomaly detection in high performance computing systems. *Eng. Appl. Artif. Intell.* **85**, 634–644 (2019)
- Borghesi, A., Molan, M., Milano, M., Bartolini, A.: Anomaly detection and anticipation in high performance computing systems. *IEEE Trans. Parallel Distrib. Syst.* **33**(4), 739–750 (2022). <https://doi.org/10.1109/TPDS.2021.3082802>
- Breiman, L., Statistics, L. B.: Random forests. *Machine Learning*, 5–32, (2001)
- Cacciarelli, D., Kulahci, M.: Active learning for data streams: a survey. *Mach. Learn.* **113**, 185–239 (2024)
- Campos, G.O., Zimek, A., Sander, J., Campello, R.J.G.B., Micenkova, B., Schubert, E., Assent, I., Houle, M.E.: On the evaluation of unsupervised outlier detection: measures, datasets, and an empirical study. *Data Min. Knowl. Discov.* **30**(4), 891–927 (2016)
- Canal, R., Hernandez, C., Tornero, R., Cilaro, A., Massari, G., Reghenzani, F., Fornaciari, W., Zapater, M., Atienza, D., Oleksiak, A., Piundefinedtek, W., Abella, J.: Predictive reliability and fault management in exascale systems: State of the art and perspectives. **53**(5), 1–32 (2020). <https://doi.org/10.1145/3403956>
- Cappello, F.: Fault tolerance in petascale/ exascale systems: Current knowledge, challenges and research opportunities. *Int. J. High Perform. Comput. Appl.* **23**(3), 212–226 (2009). <https://doi.org/10.1177/1094342009106189>
- Catillo, M., Pecchia, A., Villano, U.: Autolog: anomaly detection by deep autoencoding of system logs. *Expert Syst. Appl.* **191**, 116263 (2022). <https://doi.org/10.1016/j.eswa.2021.116263>
- Chuah, E., Jhumka, A., Narasimhamurthy, S., Hammond, J., Browne, J. C., Barth, B.: Linking resource usage anomalies with system failures from cluster log data. In: 2013 IEEE 32nd International Symposium on Reliable Distributed Systems, 111–120, (2013). <https://doi.org/10.1109/SRDS.2013.20>
- Chuah, E., Jhumka, A., Browne, J. C., Gurumdimma, N., Narasimhamurthy, S., Barth, B.: Using message logs and resource use data for cluster failure diagnosis. In: 2016 IEEE 23rd International Conference on High Performance Computing (HiPC), pp. 232–241, (2016). <https://doi.org/10.1109/HiPC.2016.035>
- Chunduri, S., Harms, K., Parker, S., Morozov, V., Oshin, S., Cherukuri, N., Kumaran, K.: Run-to-run variability on xeon phi based cray xc systems. In SC17: International Conference for High

- Performance Computing, Networking, Storage and Analysis, pp. 1–13, (2017)
- Dani, M. C., Doreau, H., Alt, S.: K-means application for anomaly detection and log classification in hpc. In: Salem Benferhat, Karim Tabia, Moonis Ali, (eds.) *Advances in Artificial Intelligence: From Theory to Practice*, pp. 201–210, Cham. Springer International Publishing. (2017)
- Das, A., Mueller, F., Siegel, C., Vishnu, A.: Desh: deep learning for system health prediction of lead times to failure in hpc. In: *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing, HPDC '18*, pp. 40–51, NY, USA. Association for Computing Machinery. (2018) <https://doi.org/10.1145/3208040.3208051>
- Di, S., Guo, H., Pershey, E., Snir, M., Cappello, F.: Characterizing and understanding hpc job failures over the 2k-day life of ibm bluegene/q system. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 473–484, (2019). <https://doi.org/10.1109/DSN.2019.00055>
- Du, M., Li, F., Zheng, G., Srikumar, V.: Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, pp. 1285–1298, NY, USA, (2017). Association for Computing Machinery. ISBN 9781450349468. <https://doi.org/10.1145/3133956.3134015>
- El Motaki, S., Yahyaouy, A., Gualous, H., Sabor, J.: Gath-geva clustering algorithm for high performance computing (hpc) monitoring. In *2019 Third International Conference on Intelligent Computing in Data Sciences (ICDS)*, pp. 1–6, (2019). <https://doi.org/10.1109/ICDS47004.2019.8942390>
- El Motaki, S., Hirchoua, B., Yahyaouy, A.: A new fuzzy MLE-clustering approach based on object-to-group probabilistic distance measure: from anomaly detection to multi-fault classification in datacenter computational nodes. *J. Ambient Intell. Humaniz. Comput.* **14**(9), 12697–12708 (2023). <https://doi.org/10.1007/s12652-022-04205-0>
- Enes, J., Expósito, R.R., Fuentes, J., Cacheiro, J.L., Touriño, J.: A pipeline architecture for feature-based unsupervised clustering using multivariate time series from hpc jobs. *Inf. Fus.* **93**, 1–20 (2022)
- Farooq, E., Borghesi, A.: A federated learning approach for anomaly detection in high performance computing. In: *2023 IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 496–500, (2023). <https://doi.org/10.1109/ICTAI59109.2023.00079>
- Farzad, A., Aaron Gulliver, T.: Unsupervised log message anomaly detection. *ICT Express* **6**(3), 229–237 (2020). <https://doi.org/10.1016/j.icte.2020.06.003>
- Farzad, A., Gulliver, T.A.: Oversampling log messages using a sequence generative adversarial network for anomaly detection and classification. *ArXiv*, abs/1912.04747, (2019). URL <https://arxiv.org/abs/1912.04747>
- Feldman, R.: Techniques and applications for sentiment analysis. *Commun. ACM* **56**(4), 82–89 (2013)
- Frank, A., Yang, D., Brinkmann, A., Schulz, M., Süß, T.: Reducing false node failure predictions in hpc. In: *2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pp. 323–332, (2019). <https://doi.org/10.1109/HiPC.2019.00047>
- Yuan, F., Liang, K., Jian, X.: Mlog: Mogrifer lstm-based log anomaly detection approach using semantic representation. *IEEE Trans. Serv. Comput.* **16**(5), 3537–3549 (2023). <https://doi.org/10.1109/TSC.2023.3289488>
- Gu, X., Wang, H.: Online anomaly prediction for robust cluster systems. pp. 1000–1011, (2009). <https://doi.org/10.1109/ICDE.2009.128>
- Guo, H., Yuan, S., Wu, X.: Logbert: Log anomaly detection via bert, (2021). URL <https://arxiv.org/abs/2103.04475>
- Guo, H., Yuan, S., Wu, X.: Logbert: Log anomaly detection via bert, (2021). URL <https://arxiv.org/abs/2103.04475>
- Gurumdimma, N., Jhumka, A., Liakata, M., Chuah, E., Browne, J.: Crude: Combining resource usage data and error logs for accurate error detection in large-scale distributed systems. In: *2016 IEEE 35th Symposium on reliable distributed systems (SRDS)*, pp. 51–60, (2016). <https://doi.org/10.1109/SRDS.2016.017>
- Han, D., Sun, M., Li, M., Chen, Q.: Ltanomaly: a transformer variant for syslog anomaly detection based on multi-scale representation and long sequence capture. *Appl. Sci.* **13**(13), 7668 (2023). <https://doi.org/10.3390/app13137668>
- Haque, A., DeLucia, A., Baseman, E.: Markov chain modeling for anomaly detection in high performance computing system logs. (2017). <https://doi.org/10.1145/3152493.3152559>
- Hickman, M., Fulp, D., Baseman, E., Blanchard, S., Greenberg, H., Jones, W., DeBardeleben, N.: Enhancing hpc system log analysis by identifying message origin in source code. In: *2018 IEEE international symposium on software reliability engineering workshops (ISSREW)*, pp. 100–105, (2018). <https://doi.org/10.1109/ISSREW.2018.00-23>
- Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997). <https://doi.org/10.1162/neco.1997.9.8.1735>
- Huang, S., Liu, Y., Fung, C., He, R., Zhao, Y., Yang, H., Luan, Z.: Hitanomaly: Hierarchical transformers for anomaly detection in system log. *IEEE Trans. Netw. Serv. Manage.* **17**(4), 2064–2076 (2020). <https://doi.org/10.1109/TNSM.2020.3034647>
- Ibidunmoye, O., Hernández-Rodríguez, F., Elmroth, E.: Performance anomaly detection and bottleneck identification. *ACM Comput. Surv.* (2015). <https://doi.org/10.1145/2791120>
- Jacob, D., Mingwei, C., Kenton, L.: and Toutanova Kristina. Pre-training of deep bidirectional transformers for language understanding, Bert (2019)
- Jauk, D., Yang, D., Schulz, M.: Predicting faults in high performance computing systems: An in-depth survey of the state-of-the-practice. (2019). <https://doi.org/10.1145/3295500.3356185>
- Jia, T., Li, Y., Zhong-Hai, W.: Survey of state-of-the-art log-based failure diagnosis. *J. Softw.* **31**(7), 1997–2018 (2020). <https://doi.org/10.13328/j.cnki.jos.006045>
- Jin, H., Papadimitriou, G., Raghavan, K., Zuk, P., Balaprakash, P., Wang, C., Mandal, A., Deelman, E.: Large language models for anomaly detection in computational workflows: From supervised fine-tuning to in-context learning. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC '24*. IEEE Press, (2024). ISBN 9798350352917. <https://doi.org/10.1109/SC41406.2024.00098>
- Sara, K.-M., Buyya, R., Ramamohanarao, K.: Performance anomaly detection using isolation?trees in heterogeneous workloads of web applications in computing clouds. *Concurrency and Computation: Practice and Experience*, pp. e5306, (2019)
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: Lightgbm: a highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, page 3149–3157, Red Hook, NY, USA, (2017). Curran Associates Inc. ISBN 9781510860964
- Kingma, D.P., Welling, M.: Auto-encoding variational bayes. *Computer Science*, (2013)
- Kirkby, J.L., Nguyen, D.H., Nguyen, D., Nguyen, N.N.: Maximum likelihood estimation of diffusions by continuous time markov chain. *Comput. Stat. Data Anal.*, 168, 107408. <https://doi.org/10.1016/j.csda.2021.107408>
- Kumar, M., Gupta, S., Patel, T., Wilder, M., Shi, W., Fu, S., Engelmann, C., Tiwari, D.: Understanding and analyzing interconnect errors and network congestion on a large scale hpc system. In: *2018 48th Annual IEEE/IFIP International Conference on Dependable*

- Systems and Networks (DSN), pp. 107–114, (2018). <https://doi.org/10.1109/DSN.2018.00023>
- Lawrence, A., Simon, L.: Annual outage analysis 2023. Technical report, Uptime Institute (2023)
- Lee, Y., Kim, J., Kang, P.: Lanobert: system log anomaly detection based on bert masked language model. *Appl. Soft Comput.* **146**, 110689 (2023). <https://doi.org/10.1016/j.asoc.2023.110689>
- Li, X., Chen, P., Jing, L., He, Z., Guangba, Yu.: Swisslog: robust anomaly detection and localization for interleaved unstructured logs. *IEEE Trans. Dependable Secure Comput.* **20**(4), 2762–2780 (2023). <https://doi.org/10.1109/TDSC.2022.3162857>
- Li, Y., Xiao, L., Feng, J., Zhang, J., Ma, Q., Yuan, Y.: Autolad: An unsupervised anomaly detection method for supercomputer logs. In: *Proceedings of the 2023 International Conference on Electronics, Computers and Communication Technology, CECCT '23*, pp. 34–37, NY, USA, (2024). Association for Computing Machinery. ISBN 9798400716300. <https://doi.org/10.1145/3637494.3637501>
- Li, Z., Zhu, Y., Van Leeuwen, M.: A survey on explainable anomaly detection. *ACM Trans. Knowl. Discov. Data* **18**(1), 1–54 (2023). <https://doi.org/10.1145/3609333>
- Scott, M.: Lundberg and Su-In Lee. A unified approach to interpreting model predictions, In *NIPS* (2017)
- Luo, Z., Hou, T., Nguyen, T.T., Zeng, H., Lu, Z.: A data-driven reinforcement learning framework. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 550–555, (2020). <https://doi.org/10.1109/INFOCOMWKSHPS50562.2020.9162664>
- MacQueen, J.B.: Some methods for classification and analysis of multivariate observations. In: *Proceedings of 5th symposium on mathematical statistics and probability*, (1967)
- Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., Zhou, R.: Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. vol. 2019-Aug, pp. 4739–4745, (2019). <https://doi.org/10.24963/ijcai.2019/658>
- Mingrui, M., Lansheng, H., Zhou, C.: Btad: a binary transformer deep neural network model for anomaly detection in multivariate time series data. *Adv. Eng. Inf.* **56**, 101949 (2023)
- Molan, M., Borghesi, A., Beneventi, F., Guarrasi, M., Bartolini, A.: An explainable model for fault detection in hpc systems. *High Performance Computing*, pp. 378–391, (2021)
- Molan, M., Borghesi, A., Benini, L., Bartolini, A.: Analysing supercomputer nodes behaviour with the latent representation of deep learning models. In *Euro-Par 2022: Parallel Processing*, (2022)
- Molan, M., Borghesi, A., Cesarini, D., Benini, L., Bartolini, A.: Ruad: unsupervised anomaly detection in hpc systems. *Future Gener. Comput. Syst.* **141**, 542–554 (2023). <https://doi.org/10.1016/j.future.2022.12.001>
- Morrow, A., Baseman, E., Blanchard, S.: Ranking anomalous high performance computing sensor data using unsupervised clustering. In: *2016 International Conference on Computational Science and Computational Intelligence (CSCI)*, pp. 629–632, (2016). <https://doi.org/10.1109/CSCI.2016.0124>
- Netti, A., Kiziltan, Z., Babaoğlu, O., Sirbu, A., Bartolini, A., Borghesi, A.: Finj: A fault injection tool for hpc systems (conference paper). *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, pp. 800–812, (2019)
- Oliner, A., Stearley, J.: What supercomputers say: A study of five system logs. In: *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, pp. 575–584, (2007). <https://doi.org/10.1109/DSN.2007.103>
- Oppenheimer, D., Ganapathi, A., Patterson, D.A.: Why do internet services fail, and what can be done about it? In: *Proceedings of the 4th Conference on USENIX Symposium on Internet Technologies and Systems-VOLUME 4, USITS'03*, pp. 1, USA, (2003). USENIX Association
- Otter, D.W., Medina, J.R., Kalita, J.K.: A survey of the usages of deep learning for natural language processing. *IEEE transactions on neural networks and learning systems*, pp. 604–624, (2021)
- Pang, G., Shen, C., Cao, L., Van Den Hengel, A.: Deep learning for anomaly detection: a review. *ACM Comput. Surv.* **54**(2), 1–38 (2021). <https://doi.org/10.1145/3439950>
- Park, C., Lee, J., Kim, Y., Park, J.-G., Kim, H., Hong, D.: An enhanced ai-based network intrusion detection system using generative adversarial networks. *IEEE Internet Things J.* **10**(3), 2330–2345 (2023). <https://doi.org/10.1109/JIOT.2022.3211346>
- Park, J.-W., Huang, X., Lee, C.-H.: Analyzing and predicting job failures from hpc system log. *J. Supercomput.* **80**(1), 435–462 (2023). <https://doi.org/10.1007/s11227-023-05482-y>
- Pelaez, A., Quiroz, A., Browne, J.C., Chuah, E., Parashar, M.: Online failure prediction for hpc resources using decentralized clustering. (2014). <https://doi.org/10.1109/HIPC.2014.7116903>
- Qi, S., Chen, J., Chen, P., Wen, P., Niu, X., Xu, L.: An efficient gan-based predictive framework for multivariate time series anomaly prediction in cloud data centers. *J. Supercomput.*, pp. 1–26, (2023)
- Ren, R., Cheng, J., Shi, H., Lei, L., Zhou, W., Wang, G., Li, C.: Failure characterization based on lstm networks for bluegene/l system logs. In *Wanling Gao, Kai Hwang, Changyun Wang, Weiping Li, Zhigang Qiu, Lei Wang, Aoying Zhou, Weining Qian, Cheqing Jin, and Zhifei Zhang, editors, Intelligent Computing and Block Chain*, pages 123–133, Singapore, (2021). Springer Singapore. ISBN 978-981-16-1160-5
- Ribeiro, M.T., Singh, S., Guestrin, C.: 'why should i trust you?': Explaining the predictions of any classifier [arxiv]. *arXiv*, pp. 15, (2016)
- Antonello Rosato, Rosa Altilio, and Massimo Panella: Recent advances on distributed unsupervised learning. *Smart Innov. Syst. Technol.* **54**, 77–86 (2016). https://doi.org/10.1007/978-3-319-33747-0_8
- Salfner, F., Lenk, M., Malek, M.: A survey of online failure prediction methods. *ACM Computing Surveys* **42**(3), 1–42 (2010). <https://doi.org/10.1145/1670679.1670680>
- Stehle, F.K., Vandelli, W., Avolio, G., Zahn, F., Fröning, H.: Resource-efficient time-series anomaly detection in dynamically-configured systems, *DeepHydra* (2024)
- Sun, J., Sun, G., Zhan, S., Zhang, J., Chen, Y.: Automated performance modeling of hpc applications using machine learning. *IEEE Trans. Comput.* **69**(5), 749–763 (2020). <https://doi.org/10.1109/TC.2020.2964767>
- Taerat, N., Leangsukun, C., Chandler, C., Naksinehaboon, N.: Proficiency metrics for failure prediction in high performance computing. In: *International symposium on parallel and distributed processing with applications*, pp. 491–498, (2010). <https://doi.org/10.1109/ISPA.2010.84>
- Tan, Y., Nguyen, H., Shen, Z., Gu, X., Venkatramani, C., Rajan, D.: Prepare: Predictive performance anomaly prevention for virtualized cloud systems. In: *2012 IEEE 32nd International Conference on Distributed Computing Systems*, pp. 285–294, (2012). <https://doi.org/10.1109/ICDCS.2012.65>
- Tian, Z., Kalupahana, L.K.S., Mohan, G.: Generative adversarial network and auto encoder based anomaly detection in distributed iot networks. In: *GLOBECOM 2020-2020 IEEE Global Communications Conference*, pp. 1–7, (2020). <https://doi.org/10.1109/GLBECOM42002.2020.9348244>
- Tuli, S., Casale, G., Jennings, N.R.: Tranad: deep transformer networks for anomaly detection in multivariate time series data. *Proc. VLDB Endow.* **15**(6), 1201–1214 (2022). <https://doi.org/10.14778/3514061.3514067>
- Tuncer, O., Ates, E., Zhang, Y., Turk, A., Brandt, J., Leung, V.J., Egele, M., Coskun, A.K.: Diagnosing performance variations in

- hpc applications using machine learning. In *High Performance Computing*, pp. 355–373, Cham, Springer International Publishing, (2017).
- Tuncer, O., Ates, E., Zhang, Y., Turk, A., Brandt, J., Leung, V.J., Egele, M., Coskun, A.K.: Online diagnosis of performance variation in hpc systems using machine learning. *IEEE Trans. Parallel Distrib. Syst.* **30**(4), 883–896 (2019). <https://doi.org/10.1109/TPDS.2018.2870403>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Llion Jones, Aidan N. Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, Gomez (2017)
- Wei, T., Peng, L., Niu, T., Zhang, H.: Detection and root cause analysis of hpc failure jobs based on feature analysis. *Front. Data Comput.* **5**(6), 94 (2023). <https://doi.org/10.11871/jfdc.issn.2096-742X.2023.06.009>
- Xia, B., Yin, J., Xu, J., Li, Y.: Loggan: A sequence-based generative adversarial network for anomaly detection based on system logs. In: Liu, Feng, Jia, Xu., Shouhuai, Xu., Yung, Moti (eds.) *Science of Cyber Security*, pp. 61–76. Springer International Publishing, Cham (2019)
- Xia, B., Bai, Y., Yin, J., Li, Y., Jian, X.: Loggan: a log-level generative adversarial network for anomaly detection using permutation event modeling. *Inf. Syst. Front.* **23**(2), 285–298 (2021). <https://doi.org/10.1007/s10796-020-10026-3>
- Xiao, R., Chen, H., Jintian, L., Li, W., Jin, S.: Allinfo: Robust diverse anomalies detection based on all log features. *IEEE Trans. Netw. Serv. Manage.* **20**(3), 2529–2543 (2023). <https://doi.org/10.1109/TNSM.2022.3224974>
- Hongzuo, X., Pang, G., Wang, Y., Wang, Y.: Deep isolation forest for anomaly detection. *IEEE Trans. Knowl. Data Eng.* **35**(12), 12591–12604 (2023). <https://doi.org/10.1109/TKDE.2023.3270293>
- Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Detecting large-scale system problems by mining console logs. In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, SOSP '09*, pp. 117–132, NY, USA, (2009). Association for Computing Machinery. ISBN 9781605587523. <https://doi.org/10.1145/1629575.1629587>
- Yang, W., Zeng, C.: A hybrid anomaly detection model based on ganomaly in cloud environment. In: *5th International Conference on Big Data and Artificial Intelligence, BDAI 2022*, pp. 51–56, (2022). <https://doi.org/10.1109/BDAI56143.2022.9862656>
- Yang, X., Wang, Z., Xue, J., Zhou, Y.: The reliability wall for exascale supercomputing. *IEEE Trans. Comput.* **61**(6), 767–779 (2012). <https://doi.org/10.1109/TC.2011.106>
- Zhang, D., Dai, D., Han, R., Zheng, M.: Sentilog: Anomaly detecting on parallel file systems via log-based sentiment analysis. In *Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems, HotStorage '21*, pp. 86–93, NY, USA, (2021). Association for Computing Machinery. ISBN 9781450385503. <https://doi.org/10.1145/3465332.3470873>
- Zhang, D., Egersdoerfer, C., Mahmud, T., Zheng, M., Dai, D.: Drill: Log-based anomaly detection for large-scale storage systems using source code analysis. In: *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 189–199, (2023). <https://doi.org/10.1109/IPDPS54959.2023.00028>
- Zhang, T., Wang, X., Li, Z., Guo, F., Ma, Y., Chen, W.: A survey of network anomaly visualization. *Sci China Inf. Sci.* **60**(12), (2017). <https://doi.org/10.1007/s11432-016-0428-2>
- Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.-G., Chintalapati, M., Shen, F., Zhang, D.: Robust log-based anomaly detection on unstable log data. page 807–817, (2019). <https://doi.org/10.1145/3338906.3338931>
- Zhang, Y., Groves, T., Cook, B., Wright, N.J., Coskun, A.K.: Quantifying the impact of network congestion on application performance and network metrics. In: *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 162–168, (2020). <https://doi.org/10.1109/CLUSTER49012.2020.00026>
- Zhao, Z., Niu, W., Zhang, X., Zhang, R., Zhenqi, Yu., Huang, C.: Trine: syslog anomaly detection with three transformer encoders in one generative adversarial network. *Appl. Intell.* **52**(8), 8810–8819 (2022). <https://doi.org/10.1007/s10489-021-02863-9>
- Zhong, Z., Fan, Q., Zhang, J., Ma, M., Zhang, S., Sun, Y., Lin, Q., Zhang, Y., Pei, D.: A survey of time series anomaly detection methods in the aiops domain. *arXiv*, (2023)
- Zou, Z., Xie, Y., Huang, K., Gongming, X., Feng, D., Long, D.: A docker container anomaly monitoring system based on optimized isolation forest. *IEEE Transactions on Cloud Computing*, 134–145 (2022)
- Zu, Y., Ghaffarkhah, A., Dang, H.-V., Towles, B., Hand, S., Huda, S., Bello, A., Kolbasov, A., Rezaei, A., Du, D., Lacy, S., Wang, H., Wisner, A., Lewis, C., Bahini, H.: Resiliency at scale: Managing google's tpuv4 machine learning supercomputer. pp. 761–774, (2024)



Yiming Lu received the BS degree in Electronic Information Engineering from the Nanchang University. He is currently working toward the Master degree in National University of Defense Technology. His current research interests include HPC and resource management.



Wei Zhang received the PhD degree from National University of Defense Technology in 2010. He is now a associate professor in the College of Computer, National University of Defense Technology. His research interests include high performance computing, parallel file system, MPI, and high speed interconnection network.



Lihua Yang received the BE and PhD degrees in computer science and technology from Huazhong University of Science and Technology (HUST) in 2016 and 2022, respectively. She is currently a postdoctoral fellow at the School of Computer Science, National University of Defense Technology. Her research interests include computer system architecture, file systems, and high performance computing.



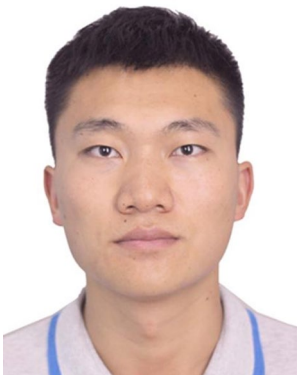
Hongyun Tian received his Ph.D. degree from the China Academy of Engineering Physics in 2023. He obtained his Bachelor's degree from Zhejiang University in 2010 and Master's degree from Beihang University in 2013. He is currently an associate researcher at the Institute of Applied Physics and Computational Mathematics(IAPCM), Beijing. His research interests include high-performance computing and the stability of large scale numerical simulations.



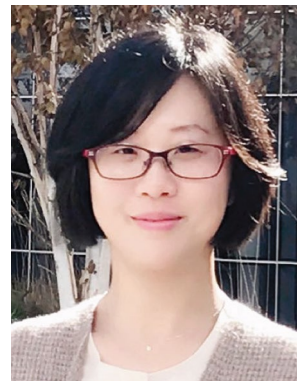
Yiqin Dai received the PhD degree in National University of Defense Technology in 2025. His research interests include high performance computing and resource management.



Yuqi Li received the BS degree from Nanchang University, China, in 2012, and MS degree from Nankai University, China, in 2017. He has worked as an engineer at National Supercomputer Centre in Tianjin (NSCC-TJ) for 11 years. His main research interests are high performance computing and AIops.



Huijun Wu received the BE and ME degrees from National University of Defense Technology in 2013 and 2015, respectively. He received the PhD degree in Computer Science and Engineering from the University of New South Wales in Australia in 2019. He is now a assistant professor in the College of Computer, National University of Defense Technology. His research interests include high performance computing, file systems, and graph neural network.



Juan Chen received the PhD degree from National University of Defense Technology in 2007. She is now a professor at National University of Defense Technology. Her research interests include supercomputer systems and energy-efficient software optimization.



Zhenwei Wu received the BE degree from Tsinghua University in 2014. He received the Master and PhD degree in Computer Science and Technology from National University of Defense Technology in 2016 and 2020, respectively. He is now an assistant professor in the College of Computer, National University of Defense Technology. His research interests include high performance computing and operating system.



Chubo Liu born in 1988. PhD. His main research interests include parallel and distributed computing, and computer architecture.



Yong Dong received the PhD degrees from National University of Defense Technology in 2012. He is now a professor in the College of Computer, National University of Defense Technology. His research interests include high performance computing, parallel storage, MPI, and resource management.